ENGINEERING PORTING REPORT

World of Warcraft Vanilla 1.12.1 on Android as a Single Application

SPP Classics functional parity using upstream CMaNGOS/playerbots components - without shipping or depending on the SPP repack

Feasibility verdict

The target is feasible as one installed Android application that imports a user-owned 1.12.1 client, starts a local CMaNGOS/playerbots realm, creates accounts, launches the Windows client through a compatibility runtime, and supervises clean shutdown. It is not a native source port of WoW.exe. The most credible architecture compiles the server and database natively for Android and runs only the closed-source 32-bit Windows client through Wine.

Audience: Implementation agent / systems engineer
Primary development target: Android x86-64 emulator on Windows
Secondary target: Physical Android ARM64 devices
Client baseline: World of Warcraft 1.12.1, build 5875
Document version: 1.0
Prepared: 1 August 2026

User-owned game files only. No Blizzard client binaries or proprietary game assets are included or recommended for distribution.

Document control and reading guide

Field	Value
Status	Agent-facing engineering reference and implementation blueprint
Scope	Vanilla 1.12.1 only; local single-player realm with CMaNGOS playerbots; one installed Android application
Out of scope	Recreating the proprietary client source, distributing client assets, connecting to Blizzard services, bypassing anti-cheat, or supporting modern retail/Classic clients
Evidence basis	Official project repositories and Android documentation, plus explicitly labeled community reports from GitHub issues and Reddit
Confidence model	Official source/documentation = authoritative for stated behavior; repository code = implementation evidence; community reports = anecdotal leads that require reproduction
Primary sequencing assumption	Prove integrated operation on x86-64 Android emulator first, then retain the native server and replace only the client runtime backend for ARM64

How the implementation agent should use this report

Read Sections 0-6 before changing architecture. Sections 7-19 specify the component contracts. Sections 20-27 define the platform sequence, acceptance gates, and issue playbook. Appendices contain configuration templates, pseudocode, diagnostic commands, community findings, and the source register.

- Treat every recommendation marked “planning estimate” as a starting hypothesis and record measured values before hard-coding it.
- Keep the client runtime behind an interface. The x86-64 and ARM64 client backends are different implementations, not build variants of one binary.
- Pin every upstream commit and database revision. Do not allow an automated agent to pull “latest” components during a reproducible build.
- Build and test one layer at a time: importer, native database, native realm server, Wine client, local login, bots, lifecycle, then ARM translation.
- Preserve the user’s source client directory unchanged. All generated realmlist/config/addon changes belong in the app-managed copy.
- A successful launch is not an acceptance result. Persistence after clean and forced shutdown, long-soak behavior, input usability, and recovery are part of the product.

Agent navigation map

Question or task	Go to
What is actually feasible?	Sections 0, 2 and 4
Which SPP behaviors should be recreated?	Section 3
What architecture should be implemented?	Sections 5 and 6
How should the single APK/import flow work?	Sections 8-10
How should CMaNGOS, MariaDB and playerbots be built?	Sections 11-14
How should WoW.exe be launched on x86-64 and ARM64?	Sections 15-16 and 21
What breaks on modern Android?	Sections 4, 8, 18 and 23
How should the emulator be configured?	Section 20 and Appendix C
What tests are release-blocking?	Section 25
How should a particular failure be diagnosed?	Section 26 issue index
Where did a claim originate?	Appendices D and E

Table of contents

Right-click and update field if this table is not populated by your viewer.

High-level section index

0. Executive summary and non-negotiable decisions

Feasibility verdict

A useful Android product is feasible, but it is not a native source port of the proprietary World of Warcraft 1.12.1 client. The implementable product is one installed Android application that imports a user-owned client, runs the open-source realm stack locally, launches the 32-bit Windows client through a Wine-compatible backend, and presents setup, accounts, bots, controls, diagnostics and recovery as one coherent experience.

0.1 The product to build

The target should be specified as an integrated local game appliance, not as "SPP Classics inside an APK." SPP Classics is a Windows repack and launcher that demonstrates a valuable workflow: select a supported expansion, install/update server content, create accounts, start MariaDB, realmd and mangosd, configure playerbots, point a matching client at the realm, and shut everything down in a controlled order. The Android application should reproduce those user-visible capabilities using upstream components and Android-native orchestration rather than redistributing the SPP package itself. [S01–S04]

The recommended runtime split is deliberately asymmetric. Compile CMaNGOS Classic, the playerbots module, the database client libraries and the local database engine for Android. Run only the original 32-bit Windows game client through Wine. On x86/x86-64 Android this removes the need for CPU instruction translation for WoW.exe; on ARM64, keep the server unchanged and replace the client backend with Wine plus an x86 translation layer. This minimizes translated CPU work, makes failures attributable, and preserves a realistic path from the emulator proof to physical devices.

0.2 Go/no-go statement

Question	Answer	Consequence
Can the closed-source 1.12.1 client be recompiled as a native Android game?	No source is available for a conventional port.	Do not scope a client-source rewrite. Treat WoW.exe and Data as imported runtime content.
Can the experience appear as one application?	Yes.	One launcher activity owns import, configuration, process supervision, controls, logs and client display. Internal native processes remain implementation details.
Can a modern production APK simply unpack and execute arbitrary ELF binaries?	Not reliably for target API 29 and above.	Production packaging must keep executable native code in APK-managed native-library locations or load components as libraries/in-process services. A target-28 proof is acceptable only as a disposable research lane. [S13]
Will stock Winlator run unchanged on the x86-64 emulator?	No.	Current upstream packaging is ARM64-oriented. Implement a client-runtime interface and an x86 backend rather than assuming an ABI rebuild is sufficient. [S09]
Can one universal APK contain x86, x86-64, ARMv7 and ARM64?	Technically possible, operationally poor.	Prefer ABI-split signed APKs or an App Bundle while maintaining one product identity and one codebase.
Can the application include Blizzard client data?	Not recommended.	Require user-selected, lawfully obtained 1.12.1 files; copy them into app-managed storage and never modify the source directory.
Should 1,000 random bots be the mobile default?	No.	Ship conservative profiles, begin with 0 or 25 bots, measure, and scale only after memory, CPU and persistence gates pass.

0.3 Non-negotiable architecture decisions

ID	Decision	Required implementation behavior
ADR-001	User-supplied client only	The APK contains no proprietary client executable, MPQ archives, cinematics, sounds, maps or other Blizzard assets.

ID	Decision	Required implementation behavior
ADR-002	One app, multiple supervised runtimes	"Single application" means one installed package and one UX. MariaDB, realmd, mangosd and Wine may execute in isolated app processes.
ADR-003	Native Android server plane	Compile the server and database for each Android ABI. Do not run the whole Windows SPP stack under Wine.
ADR-004	Wine only for the client	WoW.exe remains a 32-bit Windows x86 process. The runtime backend supplies Wine, graphics translation and, on ARM, CPU translation.
ADR-005	Client backend abstraction	x86DirectWine and armTranslatedWine implement the same start/stop/capability/diagnostics contract.
ADR-006	Managed copy, immutable source	Storage Access Framework imports into app-owned storage with a resumable journal. Patches and realmlist edits affect only the copy.
ADR-007	Pinned build graph	Every upstream repository, SQL database revision and toolchain version is pinned. No release build fetches an unbounded "latest."
ADR-008	Console/API provisioning	Accounts and administrative changes use CMaNGOS-supported console commands or a narrow local control bridge, not ad-hoc password hashing.
ADR-009	Loopback-only MVP	Database and realm services bind to loopback or Unix sockets. LAN access is a separate opt-in feature with separate threat modeling.
ADR-010	Graceful, stateful shutdown	World save/shutdown precedes realmd and database termination. Force-kill is a recovery action, never the normal stop path.
ADR-011	Measured bot tiers	Bot presets are selected from measured resource envelopes. Auction-house automation is disabled until the base realm is stable.
ADR-012	Modern Android compliance as a release gate	Foreground-service declarations, scoped storage, native-code packaging and 16 KB page-size support are tested explicitly.

0.4 Recommended milestone order

1. Prove that an imported, unmodified WoW 1.12.1 build 5875 client can be launched under an x86-compatible Wine backend inside the selected emulator image.
2. Prove CMaNGOS Classic and MariaDB natively on x86-64 Android, with no bots and no game client.
3. Join the two planes: create one local account, authenticate through 127.0.0.1:3724, enter the world on 127.0.0.1:8085, play for 30 minutes, and preserve state across a graceful restart.
4. Add importer, extraction, configuration ownership, diagnostics and recovery around the proven manual commands.
5. Add conservative playerbot tiers and 2-hour soak tests; only then add the mobile input layer and first-run polish.
6. Move to ARM64 without changing the database/server contracts; replace only the client runtime and retune graphics, memory and controls.
7. Run production-target Android compliance, 16 KB page-size, force-stop, thermal and update-migration gates before considering public distribution.

0.5 Major engineering risks at a glance

ID	Risk	L	I	Score	Primary control
R-01	Modern Android executable-code policy conflicts with a classic "unpack binaries and exec" design	5	5	25	Prototype the production packaging model early; keep target-28 work disposable.
R-02	32-bit WoW client support differs sharply between x86 emulator and ARM64 devices	5	5	25	Backend interface, capability probe and two ARM branches; do not conflate ABI with CPU translation.

ID	Risk	L	I	Score	Primary control
R-03	Winlator-derived code is not a drop-in x86 backend	5	4	20	Start with direct Wine on x86; port only reusable container/display/input components.
R-04	Client-derived map/vmap/mmap extraction is storage- and CPU-heavy	4	4	16	Checkpointed stages, free-space preflight, thermal pause, and optional deferred mmaps.
R-05	Database/core/ playerbot revisions drift and fail subtly	5	4	20	Lockfile, migration ledger, compatibility manifest and golden seed database.
R-06	Bot population exhausts memory or stalls world updates	5	4	20	Measured presets, server tick watchdog, admission control and emergency bot reduction.
R-07	Mobile controls make a technically running build unplayable	4	4	16	Treat controls/chat/targeting as release gates, not cosmetic overlays.
R-08	Process death corrupts or loses state	4	5	20	Write-ahead database settings, orderly shutdown, snapshots, crash recovery and forced-stop tests.
R-09	Graphics backend behaves differently across emulator, Adreno and Mali	5	3	15	Capability-based backend selection with WineD3D fallback and per-GPU profiles.
R-10	Licensing or proprietary-asset distribution is mishandled	3	5	15	No client assets; component notices/source offers; legal review before distribution.

Risk score is likelihood × impact on a 1–5 scale. Scores 20–25 are architecture risks and must be resolved before UX polishing; 12–19 are release-blocking unless mitigated and measured; 1–11 can be managed in the normal defect queue.

1. Scope, definitions and success criteria

1.1 Product definition

The working product name in this report is Android Local Realm. It is an Android application that turns a user-owned World of Warcraft Vanilla 1.12.1 installation into an offline or local-only experience backed by CMaNGOS Classic and playerbots. The application is responsible for data import, validation, server-data preparation, local database lifecycle, realm lifecycle, account creation, bot profiles, Wine client launch, mobile controls, logs, backup and repair.

> **Interpretation of "single APK"**
>
> The user should install and launch one application. This does not require one Linux process, one ELF file or one ABI payload. A signed APK per ABI is still a single application from the user's point of view. A universal APK should be chosen only if its installation size and native-library duplication remain acceptable.

1.2 In scope

- World of Warcraft Vanilla 1.12.1, Windows client build 5875 only.
- A local CMaNGOS Classic realm, playerbots, optional auction-house automation after stabilization, and one or more local accounts.
- Android x86/x86-64 emulator development on a Windows host, followed by ARM64 physical-device support.
- User-directed import through Android's Storage Access Framework and a managed runtime copy of the client.
- Automated generation or acquisition of server-side client-derived data only where the user has supplied the corresponding client files and the method has been legally reviewed.
- Offline operation after initial app/component installation, except when the user explicitly checks for app updates.
- Touch, keyboard, mouse and gamepad input profiles; in-game chat entry through the Android IME.
- Local backups, restore, diagnostic export, schema migration and component compatibility checks.

1.3 Explicitly out of scope

- Distribution of WoW.exe, MPQ archives, extracted proprietary assets or a preconfigured Blizzard client.
- Connecting to Blizzard's services, bypassing anti-cheat or account protections, or using modern Battle.net authentication.
- A clean-room recreation of the original game client or a native Android renderer.
- Vanilla-compatible private-server discovery, public realm hosting or automatic Internet port exposure.
- TBC, WotLK, modern Classic Era or retail clients in the first product line.
- Guaranteed compatibility with modified launchers, injected DLLs, custom patches or arbitrary private-server clients.
- Background unattended server hosting after the user has stopped the app. Runtime is user-visible and user-controlled.

1.4 Definitions

Term	Meaning in this report
Client source	The user-selected directory containing the original Windows WoW 1.12.1 client files.
Managed client	The app-owned copy used by Wine. realmlist, WTF, Interface and generated configuration changes apply here.
Server plane	MariaDB plus realmd and mangosd/WorldServer, compiled for Android.
Client plane	Wine, graphics translation, CPU translation when required, WoW.exe, virtual desktop and input mapping.
Control plane	Kotlin/Java UI, state machine, supervisor, local control sockets, diagnostics and updater.
Data pack	DBC/maps/vmaps/mmaps and SQL databases required by the chosen core revision. It must be compatible with the pinned server manifest.
SPP-equivalent behavior	A user-visible feature inspired by the SPP workflow, implemented independently using upstream components.
Ready state	Database migrations complete; realmd and world server listening; realm row correct; account provisioned; client backend ready.

Term	Meaning in this report
World-ready probe	A machine-readable signal that mangosd completed startup and can accept game sessions, not merely that its process exists.

1.5 Minimum viable product acceptance criteria

ID	Area	Release-blocking result
MVP-01	Install	One signed x86/x86-64 Android application installs without root, Termux, a separate Winlator APK or manual shell setup.
MVP-02	Import	The user selects a folder; import resumes after interruption; source files remain byte-for-byte untouched.
MVP-03	Validate	The application identifies build 5875 and rejects clearly mismatched or incomplete clients with actionable errors.
MVP-04	Prepare	Required server data and SQL schemas are prepared or imported under a pinned compatibility manifest.
MVP-05	Account	The UI creates a local account through a supported CMaNGOS command channel and stores no plaintext account password in logs.
MVP-06	Start	One Start action launches DB, realmd, world server and client in dependency order, showing stage-specific progress.
MVP-07	Play	A new character logs in, moves, fights and saves state during a continuous 30-minute session.
MVP-08	Restart	Character and world state persist across 20 graceful stop/start cycles with no recovery prompt.
MVP-09	Recover	After force-stop during play, next launch diagnoses dirty shutdown, recovers the database, and either starts safely or offers restore/repair.
MVP-10	Bots	A 25-bot preset runs for 2 hours without server tick collapse, out-of-memory termination or database corruption.
MVP-11	Controls	A touch-only user can move, steer camera, select a target, use at least 12 actions, loot, open bags, accept quests and enter chat.
MVP-12	Diagnostics	One export produces redacted app, server, database and Wine logs plus manifests and device capability data.

1.6 Quality targets after MVP

Metric	Initial target	How to measure
Warm app-to-character-select	≤ 90 seconds at 0 bots on reference x86 emulator; planning target	Timestamp from Start press to rendered character-select screen.
World server readiness	≤ 60 seconds at 0 bots; ≤ 180 seconds at 100 bots after caches are warm	Structured server-ready event, not log-text polling only.
Interactive frame rate	30 FPS floor during ordinary questing; 45 or 60 FPS optional profiles	FrameTime histogram; 95th percentile ≤ 33.3 ms for 30 FPS profile.
Server update health	No sustained update loop > 250 ms; 99th percentile recorded	Core instrumentation around world update duration.
Crash-free sessions	≥ 99% over automated 30-minute test sessions	Session telemetry stored locally; no remote telemetry required.
Import recovery	10/10 interrupted imports resume without duplicate or corrupt files	Kill at deterministic copy boundaries; compare manifest.

Metric	Initial target	How to measure
State durability	20/20 graceful cycles and 10/10 forced-stop recoveries	Database consistency check plus character-state assertions.
Storage overhead	≤ 20% above measured required content, excluding optional snapshots	Post-install directory accounting.

2. Baseline facts and compatibility matrix

2.1 Fixed protocol and content baseline

Item	Required value	Validation implication
Game family	World of Warcraft Vanilla / Classic-era original client	Do not accept TBC 2.4.3 or WotLK 3.3.5 content.
Client version	1.12.1	Display both marketing version and executable build.
Executable build	5875	The realm server build table and imported client must agree. [S05]
Windows architecture	32-bit x86	x86_64 alone is insufficient unless the Wine configuration includes 32-bit/WoW64 support.
Auth endpoint	TCP 3724 by conventional CMaNGOS configuration	Bind to 127.0.0.1 for local mode; probe before client launch.
World endpoint	TCP 8085 by conventional CMaNGOS configuration	Realm list record must advertise an address reachable from the client process. [S06]
Core	Pinned CMaNGOS Classic commit	The app manifest records Git commit and expected database revision.
Bot module	Pinned cmangos/playerbots-compatible commit	Build and SQL revisions are locked together. [S07]
Database	Pinned MariaDB Android build and schema set	Do not rely on distro package “latest.”
Client-derived data	DBC, maps; vmaps/mmaps according to selected core configuration	Validate data build/version before world start.

2.2 Platform matrix

Target	Server plane	Client execution	Primary purpose	Main blocker
Android x86 API 28 research AVD	Native x86 or x86_64 with 32-bit support	Direct x86 Wine; permissive unpack/exec research lane	Fast proof and command discovery	Not a modern production packaging model.
Android x86_64 modern AVD	Native x86_64	Wine WoW64 or bundled 32-bit compatibility	Production-policy rehearsal	32-bit guest support and executable packaging.
Android 15 x86_64 16 KB AVD	16 KB-aligned x86_64 native stack	Same as modern x86 lane	Native-library compliance	Every prebuilt .so must be 16 KB compatible. [S14]
ARM64 device with 32-bit userspace	Native arm64 server	Box86 + 32-bit Wine, or Box64/WoW64	Broad practical ARM support	Vendor/GPU variance and 32-bit availability.
ARM64 64-bit-only device	Native arm64 server	Box64 + Wine WoW64 path	Future-facing physical devices	Experimental edge cases in 32-bit Windows execution.
Mali ARM64 device	Native arm64 server	Translated Wine; conservative renderer	Compatibility fallback	No Turnip; Vulkan/OpenGL translation quality varies.
Adreno ARM64 device	Native arm64 server	Translated Wine; Turnip/DXVK candidate	Performance reference target	Driver version and thermal behavior.

2.3 ABI capability is not the same as OS bitness

The implementation must distinguish four questions: the Android application ABI, the server process ABI, the Windows client architecture, and the availability of a translator/runtime capable of the client architecture. A 64-bit Android OS may omit 32-bit Android ABI support, while Wine WoW64 can still run a 32-bit Windows application through a 64-bit Wine host if the selected Wine build supports that path. Conversely, an x86_64 emulator image may list x86_64 only and therefore reject a native x86 helper even though the CPU could theoretically execute it.

Capability probe commands to capture in every support bundle

```
adb shell getprop ro.product.cpu.abi
adb shell getprop ro.product.cpu.abilist
adb shell getprop ro.product.cpu.abilist32
adb shell getprop ro.product.cpu.abilist64
adb shell getconf PAGE_SIZE
adb shell uname -m
```

```
adb shell cat /proc/meminfo | head
```

2.4 Version lock manifest

Create one machine-readable compatibility manifest and make it the only source of truth for component selection. The UI may render friendly version names, but the supervisor must refuse to start a mixed graph.

Illustrative compatibility-manifest schema; replace placeholders at release cut

```
{
  "productSchema": 1,
  "client": {"version": "1.12.1", "build": 5875, "arch": "win32-x86"},
  "core": {"repo": "cmangos/mangos-classic", "commit": "<40-hex>"},
  "database": {"repo": "cmangos/classic-db", "commit": "<40-hex>", "requiredRevision": "<id>"},
  "playerbots": {"repo": "cmangos/playerbots", "commit": "<40-hex>", "sqlRevision": "<id>"},
  "mariadb": {"version": "<pinned>", "buildId": "android-<abi>-<hash>"},
  "wine": {"backend": "x86DirectWine", "version": "<pinned>", "buildId": "<hash>"},
  "data": {"format": 1, "clientBuild": 5875, "maps": true, "vmaps": true, "mmaps": true},
  "android": {"minApi": 28, "targetApi": "<release target>", "pageSizeCompatible": [4096, 16384]}
}
```

2.5 Evidence reliability labels

Label	Use	Agent rule
AUTH	Android official documentation, upstream project documentation, source code or license file	May define requirements, but still reproduce version-specific behavior.
CODE	Behavior inferred from repository source, scripts or build files	Treat as implementation evidence tied to a specific commit.
COMM	Reddit post, GitHub issue or user report	Use to create a test hypothesis; never make it a shipping guarantee.
EST	Planning estimate in this report	Measure on reference hardware and replace with recorded values.
DEC	Selected architecture decision	Change only through a documented ADR with impact and migration plan.

3. SPP Classics functional decomposition

3.1 What SPP Classics contributes conceptually

SPP Classics packages a CMaNGOS-based realm, playerbots and launcher automation for Vanilla, TBC and WotLK. Its release workflow downloads expansion-specific files on demand, starts server components, exposes account and bot configuration actions, and documents a client realmlist pointed at localhost. The Android project should copy the workflow model—not the Windows batch files, database credentials or binary bundle. [S01–S04]

3.2 Functional replacement matrix

SPP behavior	Android replacement	Important difference
Expansion selection	Fix product to Vanilla 1.12.1	Remove expansion ambiguity; compatibility manifest must state build 5875.
Server_Update.bat	Signed app/component updater plus migration runner	No arbitrary batch execution; staged update with rollback and hash verification.
Server_Start.bat	Android RuntimeSupervisor	Dependency graph, readiness probes, foreground notification and structured state.
Bundled MariaDB	Pinned native MariaDB build and app-private datadir	Unix socket, generated credentials, controlled recovery.
realmd.exe	Native Android realmd service/process	Loopback bind, structured health, clean termination.
mangosd.exe	Native Android mangosd service/process	World-ready signal, console command bridge, tick metrics.
Batch account menu	Account UI → console bridge	Use account commands; redact password; validate name rules.
Bot configuration files	Profile editor with safe presets	Generate config from schema; validate bounds; show projected memory/load.
Random bot generation	Cancelable first-run job	Checkpoint progress; prevent game launch until schema/generation consistency is known.
AHBot option	Advanced opt-in profile	Disabled for MVP; explicit warning and performance test requirement.
realmlist.wtf instructions	Managed-client realmlist generator	Atomic write, backup, exact loopback address.
Expansion data download	User import + extractor/data-pack validator	No proprietary asset distribution; hash/format compatibility.
Console window logs	Unified log viewer and redacted support bundle	Timestamp, process, severity, ring buffer and persisted tail.
Manual shutdown sequence	Supervisor state machine	Save world → stop world → stop realm → checkpoint DB → stop DB.
Repack defaults	Mobile-specific defaults	0/25 bots, low graphics, 720p, AHbot off, local-only network.

3.3 SPP defaults that must not be copied blindly

- Large random-bot counts: SPP documentation warns that high bot counts can cause lag. A desktop default is not a phone default. Start with no bots while validating the core, then 25, 50, 100 and measured higher tiers. [S01, S07]
- Hard-coded database credentials: launcher scripts commonly optimize for a local Windows repack, not a distributable Android threat model. Generate random app-private credentials and prefer a Unix socket.
- Open bind addresses: conventional server defaults may bind to all interfaces. Local mode must explicitly bind 127.0.0.1 and verify with /proc/net or a socket probe.
- Batch-file update semantics: copying over an old installation is unsafe when core, database and bot SQL revisions must advance together. Updates require a transactional migration ledger.
- Prebuilt data assumptions: server maps and navigation data must match the selected client/core. A directory existing is not enough; record data manifest and generation version.
- Console-driven operator knowledge: Android users should see stage-specific messages and repair actions, not be expected to interpret mangosd console output.

3.4 Product parity checklist

ID	Capability	Disposition	Definition of done
PAR-01	Import/prepare Vanilla content	Required	Importer and data pipeline complete.
PAR-02	Start/stop local realm	Required	One-action supervisor with clean persistence.
PAR-03	Create account	Required	Supported console path.
PAR-04	Configure realm name/address	Required	Local default; advanced settings hidden.
PAR-05	Generate random bots	Required after core MVP	Cancelable job and safe presets.
PAR-06	Control own alt bots	Required after bot MVP	In-game playerbot commands; optional UI helpers.
PAR-07	Auction-house automation	Deferred	Only after 100-bot soak and DB load profile.
PAR-08	Database backup/restore	Required	Consistent snapshots and restore validation.
PAR-09	View process logs	Required	Unified, filterable log stream.
PAR-10	Update components	Required for distribution	Signed, pinned, rollback-capable.
PAR-11	Multiple expansions	Not in v1	Separate future product manifests, not runtime toggles.
PAR-12	LAN realm	Deferred	Opt-in, authenticated and separately tested.

4. Legal, licensing and distribution boundary

Legal review required before distribution

This report is an engineering analysis, not legal advice. Private-server software, reverse-engineered interoperability, imported game files and app-store distribution can create copyright, trademark, contract and platform-policy questions that vary by jurisdiction. Keep the technical architecture capable of operating on user-supplied files, but obtain qualified review before releasing binaries or downloadable data packs.

4.1 Proprietary-content boundary

Asset or behavior	Recommended handling	Reason
WoW.exe and DLLs	Never bundle; import from a user-selected directory	Proprietary executable content.
Data/*.MPQ, cinematics, audio, art	Never bundle; copy only under explicit user action	Proprietary game assets.
Extracted DBC/maps/vmaps/mmaps	Generate from user files where feasible; do not host until reviewed	Derived data may retain substantial proprietary content.
realmlist.wtf	Generate inside managed copy	Configuration text created by the app.
Open-source core/server code	Build under its license and satisfy source/notice obligations	Redistributable subject to license terms.
Open-source compatibility runtime	Preserve notices, modifications and source-offer obligations	Wine/Winlator components have copyleft requirements.
Project name and UI	Use a distinct product name; descriptive references only	Avoid suggesting affiliation or endorsement.
User account credentials	Local only, redacted from logs, deletable with world data	Security and privacy obligation.

4.2 Component license inventory

Component	Observed license	Engineering action
CMaNGOS Classic	GPL-2.0 repository license	Retain copyright notices; make corresponding source and build modifications available as required. [S03]
CMaNGOS playerbots	Verify license at pinned commit	Record exact license file and compatibility with the combined server build. [S07]
Wine	LGPL family	Retain notices and satisfy relinking/source obligations for modifications.
Winlator-derived code	LGPL-2.1 in upstream repository	Track copied files and local changes; avoid treating the APK as an opaque dependency. [S09]
Box64	MIT in upstream repository	Retain notice; pin commit.
MariaDB	Mixed GPL/LGPL and library terms by component	Inventory server, connector and bundled libraries separately.
SPP Classics repository/repack	Do not assume a repository-wide redistribution grant	Use as behavior/reference evidence only until license is confirmed in writing.
WoW 1.12.1 client	Proprietary	User-provided only; no APK, expansion file or CDN distribution.

4.3 Build-time provenance record

Every release artifact should include a generated Software Bill of Materials and a human-readable notices screen. The provenance record must identify component, repository URL, commit, license file hash, local patches, compiler and build flags. A release must fail if any component lacks a license classification or if a source obligation cannot be met.

Minimum provenance entry

```
component: cmangos-mangos-classic
repository: https://github.com/cmangos/mangos-classic
```

```
commit: <40-hex>
licenseSpdx: GPL-2.0-only
licenseFileSha256: <sha256>
patches:
  - 0001-android-platform.patch
  - 0002-control-socket.patch
toolchain:
  ndk: <pinned revision>
  cmake: <pinned version>
  clang: <version>
outputs:
  - abi: x86_64
    sha256: <sha256>
  - abi: arm64-v8a
    sha256: <sha256>
```

4.4 Distribution modes

Mode	Description	Recommendation
Private development APK	Sideloaded build, no proprietary assets, detailed logs	Use for emulator and device research.
Public sideload release	Signed APKs by ABI plus source/notices	Possible after legal review, updater security and reproducible builds.
Google Play release	App Bundle, current target SDK and Play policy declarations	Treat as a separate compliance project; foreground-service and executable-runtime design may receive scrutiny.
Preloaded client bundle	APK or companion archive includes WoW data	Do not implement.
Remote downloadable client/data pack	Application fetches proprietary client or derived content	Do not implement without explicit rights and legal approval.

5. Hard platform constraints

5.1 Executable-code packaging on modern Android

Architecture blocker: writable app-home execution

For applications targeting Android 10/API 29 and later, the platform removed execute permission from the writable application home directory. A Termux-style “download or unpack ELF files into files/ and exec them” architecture is therefore not a production-safe assumption. Android recommends loading binary code embedded in the APK. [S13]

This affects CMaNGOS, MariaDB, helper tools, Wine server binaries, translation tools and any generated wrapper executable. It does not prevent the app from reading user game data. It changes where and how executable code is packaged. The agent must prove the chosen execution model on both the lowest research API and the current production target before investing in UI.

Lane	Packaging model	Use	Exit condition
Research lane	Target API 28; unpack known binaries into app-private storage and execute	Fast feasibility work and command discovery	Discard or isolate before production; never let it dictate the final update model.
Production lane A	Compile long-running components as native shared libraries invoked by small APK-packaged native launchers/services	Strongest alignment with immutable APK code	Independent process isolation, logging and shutdown proven.
Production lane B	Place PIE/ELF payloads in the APK native library area and execute from package-managed paths	Potentially simpler for existing process-oriented code	Must be prototyped against target Android versions, install modes and extractNativeLibs behavior.
Production lane C	Integrate selected runtimes in-process behind JNI	Avoids exec for some components	Only for libraries designed for embedding; do not destabilize the app process with a monolithic world server.

Recommended direction: use separate Android services/processes with small, APK-packaged native entry shims that load the real component code as shared libraries. This requires adaptation but gives Android lifecycle ownership and avoids arbitrary executable updates. Keep a second prototype of package-native executable launch only if it materially reduces Wine integration complexity.

5.2 Scoped storage and user-selected directories

Use ACTION_OPEN_DOCUMENT_TREE for folder selection and persist the URI permission. On Android 11 and later, the system picker restricts selection of storage roots, the Download root, Android/data and Android/obb. Documentation and first-run UI should direct users to a normal folder such as Documents/Games/WoW1121 rather than relying on inaccessible roots. [S15]

- Never translate a document URI into an assumed raw filesystem path. Stream through ContentResolver/DocumentFile APIs.
- Copy to app-managed storage for Wine and game I/O. Running directly from a document provider can create severe random-I/O and filename-semantic problems.
- Request only the selected tree; do not request broad legacy external-storage access as a substitute.
- Persist source URI, provider authority and a source manifest, but expect permission revocation and removable-media disappearance.
- Normalize relative paths, reject traversal, symlinks escaping the tree, device files and case-collision hazards.

5.3 Background execution and foreground-service policy

A running local realm is long-lived, CPU-active and user-visible. It should operate while the game activity is visible, under an explicit foreground service started by the user. Android 14 requires foreground-service types and type-specific declarations; specialUse exists for valid cases not covered by other types and has no runtime prerequisite, but its use must be declared and may require Play Console explanation. [S16]

Do not label indefinite realm runtime as dataSync. On Android 15, dataSync and mediaProcessing foreground services are subject to six hours total in a 24-hour period. Use user-initiated jobs or WorkManager for finite import/extraction/update operations and a visible runtime service for the active game session. [S17]

Work	Android mechanism	Rules
Import/copy	User-initiated data transfer job or foreground WorkManager job	Finite, resumable, progress notification, stop-safe.
Map extraction	Foreground worker started from visible UI	Checkpoint output; obey thermal and battery conditions; pause/resume.
Running realm and client	Foreground service + visible game activity	Started by explicit user action; persistent notification with Stop and diagnostics.
Backup	Foreground worker or user-initiated job	Acquire database snapshot/checkpoint; never copy active files blindly.
Update download	Download/WorkManager	Signed manifest, resumable and rollback-capable.
Idle app	No server processes	Respect force-stop and user stop; never silently resurrect.

5.4 Process death and memory pressure

Android may kill background processes according to process importance and memory pressure. A process existing a moment ago is not a durable guarantee. Keep the game activity visible during normal play, elevate only the legitimate runtime service, monitor child/process-service liveness and persist a small supervisor journal after every transition. [S18]

- Treat SIGKILL and app force-stop as expected fault injections, not exceptional laboratory events.
- Never rely on in-memory state to decide whether the database is consistent after restart.
- Write a boot/session ID to every process log and control message so stale sockets and orphan output are detectable.
- Use Android's onTaskRemoved/onDestroy only as best-effort hints; the database must recover without callbacks.

5.5 16 KB page-size compatibility

Android 15 supports devices configured with 16 KB memory pages. Any native library—direct or transitive—must be rebuilt and aligned appropriately, and code must not assume PAGE_SIZE equals 4096. NDK r28 and later produce 16 KB-aligned libraries by default; older toolchains require linker flags. The test plan must include an Android 15 16 KB x86_64 emulator and later an ARM64 path. [S14]

16 KB page-size release checks

```
# Runtime verification
adb shell getconf PAGE_SIZE        # expect 16384 in the compliance AVD

# APK alignment verification
zipalign -c -P 16 -v 4 app-release.apk

# For older NDK lanes only
-Wl,-z,max-page-size=16384
-Wl,-z,common-page-size=16384
```

5.6 Emulator networking is a separate address space

Inside the Android emulator, 127.0.0.1 is the Android guest itself. The Windows development host is conventionally reachable at 10.0.2.2. The integrated target should run the realm in the guest and use 127.0.0.1. Use 10.0.2.2 only for controlled A/B tests against a host-side server, and label such tests so they cannot be mistaken for a self-contained pass. [S19]

5.7 32-bit client and device evolution

WoW 1.12.1 is a 32-bit x86 Windows application. The ARM design cannot assume that the Android device exposes a 32-bit Android userspace. Maintain two tested ARM client strategies: a 32-bit Wine/Box86 path for devices that support it and a 64-bit Wine WoW64/Box64 path for 64-bit-only devices. Box64's own documentation notes that ordinary 32-bit Linux support traditionally needs Box86 or newer experimental mechanisms; therefore capability detection and device qualification are mandatory. [S10]

6. Architecture options and selected design

6.1 Option decision matrix

Option	Architecture	Strength	Weakness	Decision
A	Run the complete Windows SPP stack under Wine	Low initial integration	High translated CPU/memory; Windows MariaDB; brittle startup; licensing/repack coupling	Reject
B	Native Android MariaDB + CMaNGOS; Wine only for WoW.exe	Best resource use; clean x86→ARM boundary; upstream ownership	Requires Android port and process packaging work	Select
C	Android client connects to server on Windows host	Fastest client proof	Not offline/self-contained; emulator address confusion; no phone product	Use only as diagnostic bridge
D	Remote Internet server	No local server load	Not offline; privacy/network dependency; public-server scope	Reject for target
E	Reimplement client natively	Potential ideal UX	Multi-year clean-room protocol/render/content project; not the requested component reuse	Reject
F	Require Termux + separate Winlator	Community-proven manual route	Not one app; dependency drift; user complexity; target/API constraints	Reference only

6.2 Selected component architecture

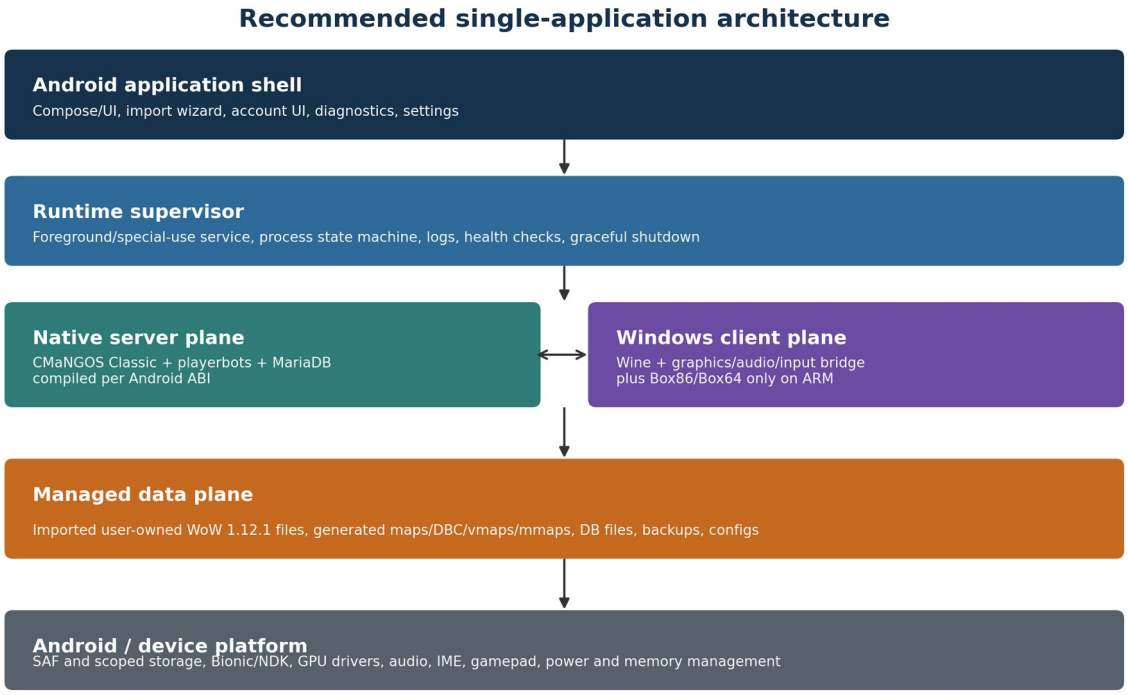

Figure 1. Selected architecture: one Android control plane supervises a native local realm and a replaceable Wine client backend.

The launcher/control plane is authoritative for state but not for world data. It communicates through narrow local interfaces: database readiness over Unix socket, realmd/world readiness over loopback and control socket, and Wine backend state over Binder/JNI or a local socket. Each component writes structured logs carrying session ID, component ID and monotonic timestamp.

6.3 Component contracts

Component	Input	Output	Invariant
ImportManager	Tree URI + requested profile	Managed client tree + manifest	Resumable journal, no source mutation, hash/size report

Component	Input	Output	Invariant
DataBuilder	Managed client + pinned extractor	Versioned DBC/maps/vmaps/mmaps	Checkpoint each stage; atomic publish of complete data set
DatabaseService	Datadir + config + migration plan	Ready socket + schema status	Exclusive owner of datadir; snapshot/recovery API
RealmService	DB endpoint + realm config	Auth-ready event	Loopback bind; exit reason and log stream
WorldService	DB endpoint + data pack + bot profile	World-ready event + control channel	Save/shutdown commands; tick/queue metrics
AccountService	Username/password/options	Account ID or structured error	Routes to core command; never logs plaintext password
ClientRuntime	Managed client + display/GPU/input profile	Game surface + runtime events	Backend-specific capability probe; clean stop
InputRouter	Touch/gamepad/IME events	Win32 keyboard/mouse input	Profile layers, pointer capture, no stuck keys
RuntimeSupervisor	Start/stop/recover requests	Observable state machine	Dependency order, timeouts, retries and durable journal
Diagnostics	Session IDs + consented export	Redacted ZIP bundle	No account passwords or raw database secrets

6.4 Runtime state machine

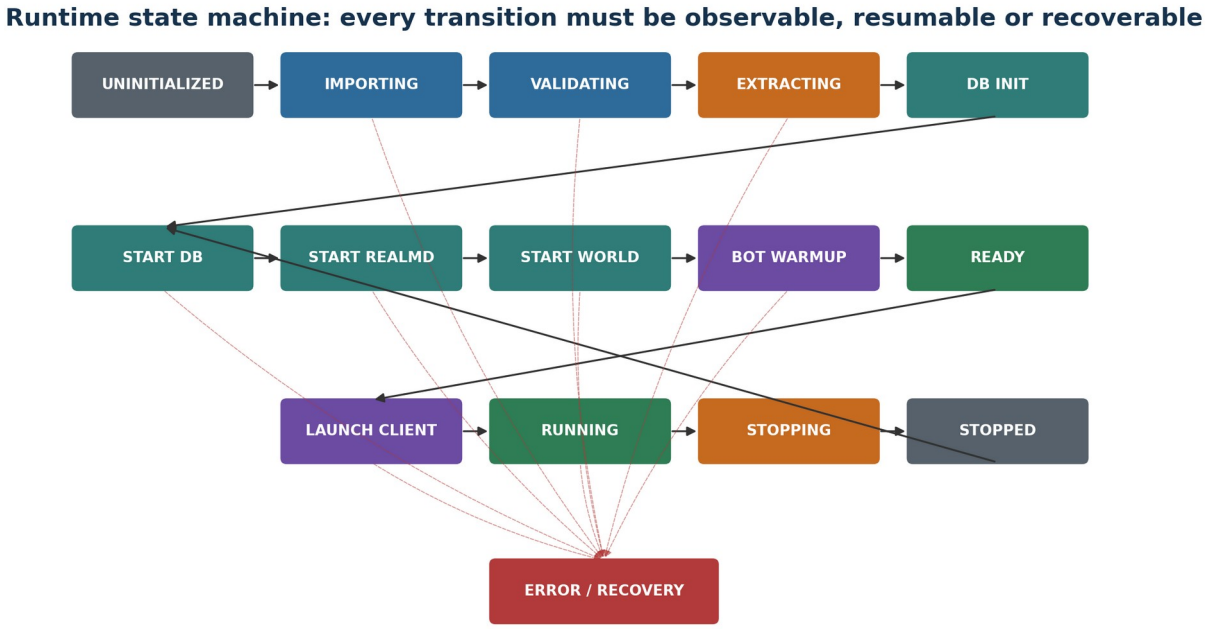

Figure 2. The supervisor publishes explicit states; it never infers readiness solely from process existence.

State	Entry condition	Exit/probe	Failure behavior
UNCONFIGURED	No valid managed client or manifest	Import completes	Remain safe; expose import/diagnostics only.
PREPARING	Import, extraction or migration active	Atomic manifest publish	Resume or rollback from job journal.
STOPPED	All runtimes stopped; data consistent	User presses Start	Run recovery check before DB start.
DB_STARTING	Database process/service launched	Socket connect + SELECT 1 + schema ledger	Stop DB; classify config/recovery/migration failure.
REALM_STARTING	DB ready; realmd launched	TCP 3724 + startup event	Stop realm; preserve DB for diagnostics or controlled stop.
WORLD_STARTING	Realm ready; world launched	World-ready control event and TCP 8085	Issue shutdown if responsive; then stop stack.
CLIENT_STARTING	World ready; Wine launched	WoW window/surface ready	Keep realm available for retry or stop by policy.
RUNNING	Client and server ready	User stop, crash or health fault	Classify client-only versus realm-fatal fault.
STOPPING	Controlled stop requested	All processes exited and DB checkpointed	Escalate TERM→timeout→KILL; mark dirty if forced.

State	Entry condition	Exit/probe	Failure behavior
RECOVERING	Dirty journal or failed consistency check	Database/core checks pass	Offer restore, repair or reset; never loop automatically.
ERROR	Unrecoverable current operation	User repair/retry/reset action	Preserve evidence and exact failed stage.

6.5 Local control protocol

Do not scrape human-readable logs as the only readiness and command mechanism. Add a small Android-specific control module or wrapper that publishes newline-delimited JSON over an app-private Unix domain socket. Keep the protocol versioned and intentionally narrow.

Illustrative local control messages; secrets should travel out-of-band or in zeroized memory

```
// World -> supervisor
{"v":1,"event":"world.ready","session":"...","build":5875,"realmId":1}
{"v":1,"event":"world.tick","p50Ms":28,"p99Ms":141,"players":1,"botsOnline":25}
{"v":1,"event":"world.shutdown.accepted","delaySec":0}

// Supervisor -> world
{"v":1,"requestId":"...","cmd":"account.create","username":"LOCAL1","passwordFd":7}
{"v":1,"requestId":"...","cmd":"server.save"}
{"v":1,"requestId":"...","cmd":"server.shutdown","delaySec":0}

// Response
{"v":1,"requestId":"...","ok":true,"code":"ACCOUNT_CREATED","accountId":42}
```

6.6 Failure-domain rules

- A client crash does not automatically terminate the realm. Offer Relaunch Client and Stop All. Time out an unattended realm only under a user-configurable, visible policy.
- A world-server crash is realm-fatal. Stop the client or return it to a controlled error surface; retain database and realmd only long enough to collect diagnostics.
- A realmd crash while an established world session exists may not immediately end gameplay, but classify the stack unhealthy and restart the whole realm at the next safe boundary.
- A database crash is realm-fatal. Do not auto-restart mangosd against a database in recovery without an explicit consistency result.
- An input-overlay crash should not kill the realm or client; reload the overlay and release all logically pressed keys/buttons.
- An updater or importer must never operate on the live managed client or live database without a snapshot/staging directory.

6.7 Recommended package/process topology

Android process/service	Contains	Reason
:ui (main)	Activities, Compose/views, settings, state observation	Keep responsive; no server or Wine native loop.
:supervisor	Foreground RuntimeSupervisor, Binder API, journals	Survives activity recreation and owns dependency state.
:database	MariaDB native launcher/library	Fault isolation and lower accidental privilege sharing.
:realm	realmd native launcher/library	Independent liveness and log identity.
:world	mangosd + Android control module	Largest server process; isolate native crash and memory.
:client	Wine backend, display bridge, translation runtime	Largest/least trusted runtime; GPU and native crash isolation.
:worker	Import/extraction/update native tools where necessary	Finite jobs; can be restarted without touching live realm.

7. Repository, build and artifact organization

7.1 Monorepo layout

Use a product monorepo that pins upstream sources as submodules or verified source archives. Keep Android adaptation patches separate from upstream trees so they can be rebased and audited. Do not copy a mutable checkout into Gradle assets by hand.

Recommended source tree

```
android-local-realm/
├── app/                          # Android UI, settings, supervisor client
├── runtime-api/                  # Binder/AIDL + state/event schemas
├── native-launchers/             # APK-packaged process entry shims
├── client-runtime-api/           # backend-neutral Wine client contract
├── client-runtime-x86/           # direct x86 Wine implementation
├── client-runtime-arm/           # Box/Wine implementation
├── server-port/
│   ├── mangos-classic/           # pinned upstream source/submodule
│   ├── playerbots/               # pinned upstream source/submodule
│   ├── classic-db/               # pinned SQL source/submodule
│   ├── mariadb/                  # pinned source/package recipe
│   └── patches/                  # ordered, documented patch queue
├── extractors/                   # map/vmap/mmap tooling and wrappers
├── packaging/
│   ├── manifests/                # compatibility and SBOM inputs
│   ├── notices/
│   └── abi/                      # x86, x86_64, arm64-v8a artifacts
├── test/
│   ├── avd/                      # reproducible emulator definitions
│   ├── fixtures/                 # non-proprietary test fixtures only
│   ├── integration/
│   └── soak/
└── docs/adr/                     # architecture decisions and migrations
```

7.2 Build graph

Stage	Input	Output	Cache key
B01 Toolchain	Pinned NDK, CMake, Ninja, JDK, Gradle	Hermetic builder image	Tool version + image digest
B02 Dependencies	Boost subset, OpenSSL/TLS if needed, zlib, MariaDB connector, libunwind	Per-ABI static/shared libraries	Source hash + flags + ABI + API
B03 MariaDB	Pinned MariaDB source/recipe	Database runtime library/launcher + utilities	Source hash + patch hash + ABI
B04 Core	CMaNGOS + playerbots + dependencies	realmd/mangosd libraries or package-native launchers	All commits + DB API + flags
B05 Extractors	Core-compatible tools	Per-ABI extraction modules/tools	Core commit + extractor options
B06 Wine x86	Pinned Wine and Android integration	x86/x86_64 client runtime payload	Wine commit + Android patch + ABI
B07 Wine ARM	Wine + Box runtime + GPU components	arm64 runtime payload	All commits + device profile data
B08 Android app	Runtime artifacts + manifests + UI	APK/AAB	Gradle lock + runtime artifact hashes
B09 Verification	Signed artifact	SBOM, notices, reproducibility report, test result	APK hash + test image IDs

7.3 Commit pinning policy

- Each upstream component is pinned to a full 40-character Git commit and a recorded repository URL.
- Submodule branches are informational only; the build uses detached commits.
- Database repositories and core/playerbot commits move together in a compatibility change request.
- A dependency-update bot may open a review, but automated release builds never advance a commit implicitly.
- Every patch has a rationale, upstream issue/link, affected ABIs and a test that would fail if the patch were removed.
- Record generated SQL and binary hashes. Re-running the same build in the same builder must produce an explained diff or a byte-identical output.

7.4 Do not cargo-cult community build workarounds

Community Android server scripts are useful evidence that native compilation can work, and they expose likely fault areas: Clang/CMake integration, missing libunwind, MariaDB library naming, version checks and insufficient memory. Some scripts create a libmariadb.so symlink, spoof an expected MySQL version, or allow multiple symbol definitions. Those are debugging clues, not production design. Reproduce the underlying link/configuration problem, then patch the build system or dependency discovery correctly. [S11–S12]

Observed workaround	Likely underlying issue	Production treatment
Symlink libmariadb.so to another filename	CMake/finder expects a desktop library name	Add an Android toolchain-aware imported target with exact library/headers.
Spoof MySQL/MariaDB version	Core configure check is overly strict or parses an absent client tool	Compile-time feature test against connector headers/API; pin supported version.
--allow-multiple-definition	Static libraries have duplicate symbols/order problems	Identify owners; change visibility, object grouping or link order. Keep the flag prohibited in release.
Disable tools globally	Extractor or utility target fails on Android	Split host tools and device tools; disable only unsupported targets and document replacement.
Build directly inside Termux	Convenient on-device environment	Use as proof only; production artifacts come from hermetic cross-compilation.
Pull latest repo during install	Simplifies a hobby script	Replace with signed manifest and pinned source/binary hashes.

7.5 Build variants

Variant	Target/API	ABIs	Purpose	Distribution
researchLegacy	Target 28	x86, x86_64 as available	Rapid unpack/exec and Wine feasibility	Internal only; prominent watermark/diagnostic flag.
debugModern	Current target	x86_64, arm64-v8a	Production packaging, lifecycle and policy tests	Internal/test lab.
releaseX86_64	Current target	x86_64	Emulator/reference artifact	Sideload/CI only unless useful to users.
releaseArm64	Current target	arm64-v8a	Primary physical-device release	Signed release.
releaseUniversal	Current target	x86_64 + arm64-v8a; optional x86	Single downloadable APK when size is acceptable	Optional, generated from same manifests.

7.6 Artifact manifest and startup verification

At application startup, verify the immutable runtime artifacts against a compiled-in or APK-signed manifest. Verify imported mutable data against its own manifest only when needed; full hashing on every launch is too expensive. Use size/mtime plus sampled hashes for quick checks and a user-triggered deep verification.

Runtime artifact manifest concept

```
runtimeArtifacts:
  - id: mangosd-x86_64
    apkPath: lib/x86_64/libmangosd_runtime.so
    sha256: <hash>
    pageSizes: [4096, 16384]
  - id: wine-x86
    apkPath: lib/x86/libwine_runtime.so
    sha256: <hash>
    executablePolicy: apk-native
schemas:
  compatibility: 3
  supervisorJournal: 2
  databaseMigration: 14
signing:
  manifestKeyId: release-2026-01
  signature: <base64>
```

8. Android application shell and runtime packaging

8.1 UI information architecture

Screen	Primary functions	Must show
Home	Start, Stop, Relaunch Client, current realm status	Exact stage, bot tier, last safe save, prominent Stop.
Setup	Select/import client, prepare server data, storage location	Build result, required/free space, progress and resume state.
Accounts	Create/change/delete local accounts	Local-only warning, validation, no password disclosure.
Bots	Select tier and behavior profile	Projected load, currently online bots, generation status.
Display & Controls	Resolution, renderer, FPS cap, overlay/gamepad profiles	Compatibility warnings and reset-to-safe-mode.
World	Realm name, rates, advanced server options	Defaults first; unsafe options grouped and reversible.
Backups	Create, restore, export metadata	Consistency state, size, date, app/core version.
Diagnostics	Live logs, capability report, repair actions, support bundle	Redaction notice, component/session filter.
About & Licenses	Version, components, source/notices	Exact commits/build IDs and legal notices.

8.2 Runtime service contract

Expose one bound foreground RuntimeService to the UI. The UI sends intents or Binder calls; it does not spawn or signal native processes directly. RuntimeService owns the journal, acquires a partial wake lock only during user-visible startup/shutdown or finite critical sections, posts the foreground notification immediately, and publishes a StateFlow/observable state snapshot.

Backend-neutral supervisor API

```
interface RuntimeController {
  suspend fun preflight(): PreflightReport
  suspend fun start(profileId: String): OperationId
  suspend fun stop(mode: StopMode = GRACEFUL): OperationId
  suspend fun relaunchClient(): OperationId
  suspend fun executeAdmin(command: AdminCommand): AdminResult
  fun observeState(): Flow<RuntimeSnapshot>
  fun observeLogs(filter: LogFilter): Flow<LogEvent>
}

data class RuntimeSnapshot(
  val sessionId: UUID?,
  val phase: RuntimePhase,
  val database: ComponentState,
  val realm: ComponentState,
  val world: ComponentState,
  val client: ComponentState,
  val progress: Progress?,
  val recoverability: Recoverability
)
```

8.3 Foreground notification

- Title: “Local realm running” or exact startup stage.
- Content: realm profile, 1 local player, N bots, client state and elapsed session duration.
- Actions: Return to game, Relaunch client when applicable, Save and Stop, Diagnostics.
- Ongoing only while the stack is active; remove after database shutdown and journal commit.
- A forced Stop action must not wait for the UI activity to exist.
- Never display account password, database credential or full source path in notifications.

8.4 Native runtime packaging experiments required before feature work

ID	Experiment	Method	Decision enabled
PKG-01	Can an APK-packaged native launcher execute from nativeLibraryDir on each target API?	Simple hello/exit process; capture SELinux denial and linker namespace.	Required for process-oriented production lane.
PKG-02	Can the launcher dlopen a large server library and isolate it in android:process?	Start/stop dummy loop; deliberate native crash.	Proves library-backed component model.
PKG-03	Can Wine runtime modules locate each other without writable executable extraction?	Launch wineboot/notepad equivalent from packaged modules.	Highest-risk client packaging proof.
PKG-04	Can mutable Wine prefix/data remain app-private while code remains immutable?	Create prefix, relaunch, verify registry and file writes.	Separates code and state.
PKG-05	Can native payload updates be delivered only through signed APK update?	Install upgrade, retain data, verify code hash changes.	Production update contract.
PKG-06	Does every .so run on 4 KB and 16 KB page-size AVDs?	Startup and 30-minute smoke on both.	Release gate.

8.5 App directory model

Illustrative app-private data layout

```
noBackupFilesDir/
├── runtime-state/
│   ├── supervisor-journal.json
│   ├── component-pids-or-tokens/
│   └── last-session/
├── database/
│   ├── datadir/
│   ├── run/mariadb.sock
│   └── backups/
├── server/
│   ├── config/
│   ├── data/<manifest-id>/
│   └── logs/
├── wine/
│   ├── prefix/
│   ├── cache/
│   └── logs/
├── client/
│   ├── active/                # managed client copy
│   ├── staging/
│   └── import-journal/
└── manifests/

cacheDir/
├── extraction-work/
├── update-downloads/
└── diagnostic-staging/
```

Use noBackupFilesDir for large runtime data that should not be silently included in Android Auto Backup. Provide explicit in-app backups. Mark cache content safely disposable. Ensure every path has a quota/cleanup owner.

8.6 Permissions and exported components

Item	Policy
Storage	Use SAF grants; avoid MANAGE_EXTERNAL_STORAGE unless a legally and technically justified separate distribution requires it.
Network	INTERNET is needed for local sockets and optional updates; local-only mode must not imply external traffic.
Foreground service	Declare base permission and exact type permission; document specialUse subtype if selected.
Wake lock	Use only for bounded startup/shutdown/extraction critical sections, not entire idle sessions by default.
Notifications	Request notification permission where required; explain that runtime control depends on it.

Item	Policy
Services/providers/receivers	android:exported=false unless a documented external contract exists.
Debug interfaces	Compile out or gate by debug signature. No unauthenticated TCP admin console.
Native debugging	Disable debuggable and strip symbols in release; retain separate symbol archives by build ID.

9. Client import, validation and managed-copy design

9.1 Import user journey

1. Explain that the user must supply an existing World of Warcraft 1.12.1 client directory and that the app will make a private working copy.
2. Open the system folder picker and persist read permission to the selected tree.
3. Perform a fast structural scan and executable/build inspection without copying everything.
4. Calculate source bytes, required destination bytes, extraction estimate, database estimate, snapshot margin and a safety reserve.
5. Present a validation report. Reject unsupported builds; allow a clearly labeled “advanced diagnostic import” only for missing optional files, not build mismatch.
6. Copy into a staging directory using a durable journal, per-file temporary names and bounded parallelism.
7. Verify copied content, write the managed-client manifest, apply only app-owned configuration changes, then atomically publish staging as active.
8. Release or retain source URI permission according to the user’s choice; the runtime never depends on source availability after a complete import.

9.2 Fast validation checklist

ID	Check	Class	Failure outcome
VAL-01	WoW.exe exists and is readable	Required	Missing executable.
VAL-02	PE architecture is IMAGE_FILE_MACHINE_I386	Required	Wrong client architecture.
VAL-03	Version resource/build indicates 1.12.1 build 5875	Required	Unsupported build; do not “try anyway” in normal mode.
VAL-04	Data directory exists with base MPQs	Required	Incomplete install.
VAL-05	At least one supported locale directory/locale MPQ exists	Required	Unsupported/incomplete locale.
VAL-06	No source path collision after case-folding	Required	Copy to case-sensitive/case-normalized target may overwrite.
VAL-07	No traversal/special-file entries	Required	Unsafe provider/source.
VAL-08	File count and total bytes within configured sanity bounds	Required	Likely wrong root or recursive mount.
VAL-09	WTF/Interface present	Optional	Create defaults; copy addons if compatible.
VAL-10	Known custom DLL/injector/launcher artifacts	Warning	Safe mode excludes or disables injected additions.

9.3 Build identification strategy

Do not depend on one hard-coded SHA-256 for WoW.exe because legitimate locale or distribution variants may differ. Parse the PE header and version resource where possible, inspect known build strings, and corroborate against the client’s expected data layout. Maintain a signed allowlist of known executable hashes as high-confidence evidence, not the sole criterion. Unknown hash + confirmed build may proceed with a warning; wrong build must not.

Client identity model

```
data class ClientIdentity(
  val peMachine: Int,
  val fileVersion: String?,
  val productVersion: String?,
  val detectedBuild: Int?,
  val sha256: String,
  val localeCandidates: Set<String>,
  val confidence: IdentityConfidence,
  val warnings: List<ValidationWarning>
)

acceptNormal = peMachine == I386 && detectedBuild == 5875 && dataLayout.valid
acceptDiagnostic = false  // never bypass build mismatch in ordinary UI
```

9.4 Storage preflight calculation

Calculate from the selected source instead of publishing a fixed number. The initial planning formula should reserve the full managed copy, generated server data, database, Wine prefix/caches, one consistent database snapshot and 20% working margin. Present exact measured and estimated values in GiB.

Initial storage calculation; replace estimates with observed per-build statistics

```
requiredBytes = sourceBytes
              + estimatedExtractedDataBytes
              + estimatedDatabaseBytes
              + estimatedWinePrefixAndCacheBytes
              + minimumSnapshotBytes
              + max(2 GiB, ceil(0.20 * subtotal))

proceed only if allocatableBytes >= requiredBytes
warn if allocatableBytes < requiredBytes + 2 GiB
```

Category	Planning estimate	Notes
Managed WoW client	4–8 GiB	Measure selected source; custom clients can be larger.
DBC/maps/vmaps/mmaps	1–4 GiB	Core/extractor options drive size; mmaps can dominate.
MariaDB data	0.5–2.0 GiB initially	Bot characters, logs and AH data increase it.
Wine prefix/runtime state	0.5–1.5 GiB	Code may live in APK; mutable prefix/cache remains.
Backup and staging	1–4 GiB	At least one DB snapshot plus interrupted-operation staging.
Total practical free-space request	8–16 GiB	Planning range only; app must compute actual value.

9.5 Resumable copy journal

The journal must be append-safe and recoverable if the process is killed between bytes, file close, hash calculation and directory publish. A SQLite journal is preferable to one repeatedly rewritten JSON file when the source has thousands of entries.

Journal field	Purpose
import_id / schema_version	Identify operation and permit journal migrations.
source_tree_uri + persisted flags	Reopen source when permission remains.
source_fingerprint	Detect that the selected tree changed between resume attempts.
relative_path	Stable normalized key; never a raw external path.
expected_size / mtime / hash	Validation evidence at chosen assurance level.
bytes_copied / temp_name	Resume or safely restart an individual file.
state	DISCOVERED, COPYING, COPIED, VERIFIED, FAILED, SKIPPED.
attempt / last_error	Bound retries and expose actionable failure.
fsync_marker	Record that content and parent directory publication reached durable boundary.

9.6 Copy algorithm

Resumable managed-copy algorithm

```
for each normalized source entry in deterministic order:
    reject unsafe type/path/case collision
    if journal says VERIFIED and quick source fingerprint still matches:
        continue
    create destination ".<name>.partial.<importId>"
    stream with 1–4 MiB buffer; update progress every 8–32 MiB
    flush and fsync file
    verify size; hash all critical files and sampled/all remaining files by policy
    atomically rename partial file to final name
    fsync parent directory where supported
    journal state = VERIFIED
```

```
when all required entries are VERIFIED:
    write client-manifest.json.partial
    fsync + rename to client-manifest.json
    atomically switch active pointer/directory
    mark import COMPLETE
```

9.7 Managed-client modifications

File/area	Action	Rollback
realmlist.wtf	Write exact local realm address in compatible locale path; normalize line endings	Keep app-generated previous copy and regenerate from settings.
WTF/Config.wtf	Apply safe renderer/resolution/sound defaults only after first Wine prefix is ready	Track each app-owned key; preserve user keys.
WTF/Account	Do not pre-populate account passwords	Not applicable.
Interface/AddOns	Optionally install vetted 1.12-compatible control addon into app namespace	Manifest-owned files removable; user addons preserved.
WoW.exe / DLLs / MPQs	Never patch in MVP	Any future binary patch requires separate legal/security review and reversible copy.
WDB/Cache	May clear on repair when known safe	Warn that caches rebuild.

9.8 Re-import and update behavior

- A re-import builds a new staging tree; it never overlays the active client in place.
- Offer “preserve settings and addons” as a three-way merge: old managed copy, new source copy, app-owned defaults.
- Do not migrate WDB/cache blindly across incompatible client identities.
- Keep one previous active manifest until the new client reaches character-select once; then allow cleanup.
- Source deletion after import has no effect. Managed-data deletion is an explicit destructive action with size preview.

10. Client-derived server data extraction

10.1 Required outputs and sequencing

Stage	Output	Required for	Mobile treatment
D01 DBC extraction	dbc/	Core definitions/content metadata	Small/fast relative to geometry; validate build first.
D02 Map extraction	maps/	Terrain/map access	Required before first world start.
D03 VMap extraction/assembly	vmaps/	Line-of-sight/collision	Strongly recommended; missing/bad data causes gameplay/pathing defects.
D04 MMap generation	mmaps/	Movement/pathfinding	CPU/storage heavy; checkpoint by map/tile; allow deferred completion only with clear degraded-mode policy.
D05 Manifest	data-manifest.json	Compatibility/startup validation	Publish only after required outputs pass structural checks.

10.2 Host-generated versus on-device data

Model	Advantages	Disadvantages	Recommendation
On-device extraction from user client	Self-contained; provenance tied to user files; no derived-data hosting	Long, hot, storage-heavy; extractor Android port required	Preferred default if stable.
Desktop companion extraction	Faster on PC; easier tools	Breaks one-app setup; transfer UX; companion maintenance	Optional advanced path, not MVP dependency.
Download pre-generated data pack	Fast setup	Legal/distribution risk; version drift; large bandwidth	Do not ship without explicit rights/review.
Bundle generated data in APK	Simple runtime	Very large APK; proprietary/derived-content risk; one fixed core	Reject.

10.3 Extraction job design

- Run extraction as an explicit finite foreground job, not hidden during the first game launch.
- Publish stage, map/tile index, processed/total count, bytes written, estimated remaining work based on observed throughput, device temperature status and pause reason.
- Write each output to a versioned staging directory; publish by atomic active-manifest switch only after validation.
- Checkpoint mmaps at map/tile granularity. On resume, verify existing output before skipping it.
- Limit worker threads based on big-core count, memory and thermal state; default to min(4, max(1, availableProcessors/2)) as an estimate, then tune.
- Pause or reduce concurrency on severe thermal status; never continue until Android kills the process or storage fills.
- Keep the screen optional. The foreground notification must allow pause/cancel and return to setup.

10.4 Data validation

Check	Failure prevented
Manifest clientBuild == 5875	Starting a core against wrong client-derived data.
Extractor build ID matches pinned core family	Silent format drift between core and tools.
Expected directory and index files exist	False “complete” after interrupted stage.
File counts and non-zero size bounds by map	Truncated or empty output.
Sample/open parser validates headers	Wrong endian/version/corrupt output.
MMap tile coverage report records gaps	Unexplained bot/pathfinding failures.
Final directory is read-only to normal runtime where feasible	Accidental mutation by world process.

10.5 Degraded data modes

Do not silently start without collision or navigation data. If the core can technically run without vmaps/mmaps, expose a named diagnostic mode with a persistent warning and bots disabled. Community reports associate missing or poor navigation data with line-of-sight, water pathing and movement failures; use those reports as explicit regression scenarios. [S26]

Mode	Allowed content	Restrictions
Normal	DBC + maps + validated vmaps + expected mmaps	Bots allowed according to tier.
No-mmaps diagnostic	DBC + maps + vmaps	Bots off; no acceptance testing; red warning; logs tag degraded mode.
Minimal core diagnostic	DBC + maps, if core supports it	Developer only; no user release.
Incomplete/corrupt	Any required set fails validation	World start blocked; repair/re-extract only.

10.6 Extraction performance record

Store observed throughput by device profile and stage so later estimates are evidence-based. Capture source bytes read, output bytes, wall time, CPU time, peak RSS, thermal throttling duration and retry count. Never promise a fixed duration in the UI before at least one comparable observation exists.

11. CMaNGOS Classic Android port

11.1 Upstream baseline and support boundary

CMaNGOS Classic is the appropriate upstream core for the requested Vanilla build. Its official installation material documents desktop operating systems, not Android; therefore Android is a downstream platform port that must own its toolchain, patches and tests. The official realm-list source recognizes Vanilla build 5875, which must remain a pinned compatibility invariant. [S03, S04, S05]

11.2 Porting strategy

1. Build the unmodified pinned core for a supported desktop target and run its tests/tools. This is the behavioral reference.
2. Create an NDK toolchain build for the smallest server target first: realmd without playerbots or extraction tools.
3. Resolve platform feature tests explicitly. Do not let CMake mistake Android for generic Linux where APIs differ.
4. Add mangosd with playerbots disabled. Reach database, load DBC/maps and publish world-ready.
5. Add the Android control/logging module and graceful stop path.
6. Enable the pinned playerbots module and its SQL only after a zero-bot world is stable.
7. Build each ABI from the same source/patch set and run protocol/database golden tests before device tests.

11.3 Likely portability areas

Area	Expected issue	Port rule
Filesystem paths	Desktop relative paths, executable directory assumptions, case sensitivity	Inject absolute app-owned paths; no chdir-based global contract.
Signals/console	Interactive stdin console and POSIX signal behavior differ under Android services	Add control socket; preserve signal handlers only as fallback.
Process model	Fork/daemon assumptions	Run foreground in managed Android process; no daemonization.
Networking	Interface enumeration/bind defaults	Explicit loopback address and IPv4/IPv6 policy.
Time/timers	Clock source or sleep behavior under suspend	Use monotonic clocks; instrument long update gaps.
Thread names/priorities	pthread APIs and scheduler permissions	Best-effort names; avoid privileged priority assumptions.
Crash dumps	Desktop core dumps unavailable/restricted	Use tombstones/native crash reporting and separate symbols.
Locale/encoding	Desktop locale discovery	Force deterministic UTF-8/config behavior where supported.
Dynamic modules	dlopen paths and plugin layout	Prefer static script modules or APK-packaged libraries with fixed manifest.
Dependencies	Desktop Boost/MySQL/OpenSSL discovery	Imported CMake targets built for Android; no host contamination.
Memory allocator	Desktop defaults may fragment under bots	Measure; consider supported allocator only after baseline.
Page size	Hard-coded 4096 or mmap alignment	Use sysconf/getpagesize; 16 KB CI.

11.4 Build configuration

Illustrative bootstrap command; option names must be reconciled with the pinned commit

```
cmake -S server-port/mangos-classic -B build/android-x86_64 \
  -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE=$ANDROID_NDK/build/cmake/android.toolchain.cmake \
  -DANDROID_ABI=x86_64 \
  -DANDROID_PLATFORM=android-28 \
  -DBUILD_PLAYERBOTS=OFF \
  -DBUILD_EXTRACTORS=OFF \
  -DBUILD_TOOLS=OFF \
  -DBUILD_SHARED_LIBS=OFF \
  -DANDROID_LOCAL_REALM=ON \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo

cmake --build build/android-x86_64 --target realmd mangosd -j <bounded>
```

Prefer static linkage for small dependency libraries when licensing and size permit, but do not force the entire product into one process. If the production execution model loads mangosd as a shared library, define a narrow C ABI entry point and keep third-party symbol visibility hidden.

11.5 Android adaptation patch queue

Patch	Purpose	Required test
0001-android-toolchain-detection	Correct feature macros, library discovery and unsupported targets	CMake configure contains no host include/library path.
0002-app-path-provider	Supply absolute config/data/log paths from launcher	Start from arbitrary cwd and read only app paths.
0003-no-daemon-no-interactive-console	Foreground service mode	No stdin required; process remains supervised.
0004-structured-log-sink	Forward logs with session/component metadata	Log ordering and crash-tail test.
0005-local-control-socket	Readiness, account and shutdown commands	Protocol version/error/fuzz tests.
0006-android-shutdown-hooks	Save/checkpoint and bounded exit	20 graceful cycles and forced escalation.
0007-metrics-hooks	World tick, players, bots, queues, RSS	Metric values under scripted load.
0008-page-size-cleanups	Remove 4 KB assumptions	4 KB and 16 KB emulator tests.
0009-library-entry-shim	Optional process library entry point	Native crash isolated to :world process.

11.6 Configuration ownership

Generate configuration from a typed schema and preserve a rendered copy for diagnostics. Do not expose every mangosd.conf key in the normal UI. Divide settings into product-owned, user-safe and developer-only groups.

Class	Examples	Edit policy
Product-owned	DB endpoint, bind IP, data/log paths, realm ID, control socket	Generated only; UI cannot override arbitrary text.
User-safe	Realm name, XP/drop rates within bounds, bot tier, locale, log level	Typed UI with validation and reset.
Advanced	Selected playerbot behavior/AH options	Schema-backed editor with warning and profile export.
Developer-only	Threading/debug/DB pool internals, arbitrary config text	Debug build or explicit expert unlock; never part of support guarantee.

11.7 Readiness and health

- Process started is not ready.
- realmd readiness requires successful schema checks, realm table load and listening socket.
- world readiness requires all required databases, client-derived data, scripts and playerbot SQL to load, followed by the core's normal ready point.
- Publish a structured ready event from the Android module and corroborate with a socket probe.
- During RUNNING, emit world update-duration histograms and a heartbeat. A missing heartbeat is not immediately fatal if the world thread is busy; combine it with process and socket state.
- Classify start failures by stable error codes such as DB_CONNECT, DB_REVISION, DATA_MISSING, DATA_BUILD, PORT_IN_USE, BOT_SQL and ASSERT.

12. MariaDB integration, migrations and backup

12.1 Why keep a real local database

CMaNGOS is designed around MySQL/MariaDB schemas and SQL migrations. Replacing the database layer with SQLite would turn the effort into a broad core rewrite and create ongoing incompatibility with upstream database and playerbot SQL. Use a native MariaDB server, but constrain it as an app-private embedded service.

12.2 Database service invariants

- The datadir is owned exclusively by DatabaseService; no other component copies or edits live table files.
- Prefer an app-private Unix domain socket and skip TCP listening. If connector limitations force TCP in a research build, bind only 127.0.0.1 and verify.
- Generate a random database administration secret and a separate least-privilege core user on first initialization.
- Do not expose the database root account to the UI or write secrets to mangosd logs/support bundles.
- Use an explicit clean-shutdown marker and inspect MariaDB recovery output after dirty stop.
- Run schema migrations with the world and realm stopped and a verified pre-migration snapshot available.

12.3 Initial mobile-oriented configuration

Illustrative configuration; benchmark durability/performance before fixing values

```
[mariadbd]
datadir=<app>/database/datadir
socket=<app>/database/run/mariadb.sock
skip-networking=1
skip-name-resolve=1
character-set-server=utf8mb4
collation-server=utf8mb4_general_ci
max_connections=24
performance_schema=OFF
innodb_buffer_pool_size=<computed profile, e.g. 256M–768M>
innodb_log_file_size=<measured/pinned>
innodb_flush_log_at_trx_commit=1
sync_binlog=0
log_error=<app>/database/mariadb-error.log
pid_file=<app>/database/run/mariadb.pid
secure_file_priv=<app>/database/export

[client]
socket=<app>/database/run/mariadb.sock
```

The buffer-pool range above is a planning estimate, not a default for every device. Compute a profile from total RAM and selected bot tier, cap it conservatively, and measure peak resident memory of the complete stack. Keep durability settings strong until profiling proves a specific change safe.

12.4 Initialization flow

1. Create datadir and restrictive permissions inside app-private storage.
2. Initialize system tables using the pinned MariaDB method in an isolated initialization mode.
3. Start on Unix socket only and wait for a successful authenticated query.
4. Generate app-internal database credentials using a cryptographically secure RNG; store encrypted with Android Keystore where practical.
5. Create core user restricted to localhost/socket and grant only required schema privileges.
6. Create/import realm, characters, logs and world databases from pinned seed/migration inputs.
7. Run compatibility assertions and write database-manifest.json only after all statements commit.
8. Stop cleanly, checkpoint and create a compact golden-initialized snapshot for repair/reset if distribution terms allow.

12.5 Migration ledger

Field	Description
migration_id	Globally unique ordered identifier.
component	core, classic-db, playerbots, product.
source_commit / target_commit	Compatibility transition.
sql_sha256	Exact SQL input integrity.
started_at / finished_at	Audit and interrupted-migration detection.
status	PENDING, APPLIED, ROLLED_BACK, FAILED.

Field	Description
pre_snapshot_id	Required restore point for destructive migration.
app_build_id	Release that applied it.
result_digest	Selected revision-table values after migration.

12.6 Backup model

Backup type	Method	Use
Quick automatic snapshot	Database-consistent logical dump or snapshot after clean stop	Before app/core update and periodically after successful sessions.
Manual named backup	Consistent dump + manifests + configs; optional compression	User restore point.
Crash recovery state	InnoDB recovery plus journal; not called a backup	Restore service after dirty stop without losing latest committed data.
Support export	Schema/revision metadata and redacted diagnostics, not full character DB by default	Issue investigation.
Full world export	Explicit user action, encrypted archive recommended	Migration between devices or archival.

Copying the datadir while MariaDB is live is not an acceptable backup. Either stop cleanly or use a database-supported consistent backup/dump mechanism. Validate every created backup by reading its manifest and, in CI, regularly restore into a fresh datadir and run server startup assertions.

12.7 Database failure categories

Code	Symptom	First action
DB-INIT	System tables or datadir absent	Resume idempotent initialization; never import world SQL twice blindly.
DB-LINK	Native linker cannot find libmariadb/dependency	Inspect DT_NEEDED and APK ABI packaging; do not symlink randomly.
DB-SOCKET	Server starts but core cannot connect	Verify socket path length/permissions and connector Unix-socket option.
DB-REVISION	Core reports required update/revision mismatch	Stop; compare compatibility manifest and migration ledger.
DB-RECOVERY	Dirty shutdown recovery fails or loops	Preserve datadir, export error log, offer restore; no repeated forced starts.
DB-OOM	Database killed under bot load	Reduce bot tier/buffer pool; capture complete memory profile.
DB-FULL	Disk full during transaction/migration	Stop stack; free space; restore/preflight before retry.
DB-CORRUPT	Table/page corruption reported	Read-only evidence copy, supported recovery, then restore; do not auto-repair destructively.

13. Playerbots and auction-house load management

13.1 Feature baseline

The CMaNGOS playerbots module can populate the open world, battlegrounds and arenas, use alternate characters as bots, assist with PvE and expose detailed behavior configuration and commands. That breadth is why it is valuable, but it also makes it the largest server-side performance and correctness variable. [S07]

13.2 Progressive enablement

Profile	Random bots	Alt bots	AH automation	Purpose
B0 Core validation	0	0	Off	Database/core/client integration and persistence baseline.
B1 Low	25	Up to 2	Off	Default mobile proof and touch/control testing.
B2 Medium	50	Up to 4	Off	Reference 6–8 GB device test.
B3 High	100	Up to 8	Off	Qualified hardware only; 2-hour soak required.
B4 Extreme	150–300 measured	Measured	Optional	Developer/advanced; not a universal promise.
Custom	Validated bound	Validated bound	Explicit opt-in	Show projected and observed resource use.

> **Do not inherit a desktop bot count**
>
> SPP documentation and community experience indicate that large populations can add substantial startup and runtime load. Ship the Low profile only after zero-bot stability, and never describe a 1,000-bot configuration as a normal phone target. [S01, S27]

13.3 Bot profile schema

Illustrative typed profile, not literal upstream key names

```
{
  "id": "mobile-low-v1",
  "randomBots": {"enabled": true, "target": 25, "min": 20, "max": 30},
  "altBots": {"maxOnline": 2},
  "auctionHouse": {"enabled": false},
  "generation": {"batchSize": 5, "yieldMs": 250, "resume": true},
  "behavior": {
    "allowBattlegrounds": false,
    "allowArenas": false,
    "lootStrategy": "conservative",
    "reactionBudgetMs": 50
  },
  "admission": {
    "maxWorldP99Ms": 250,
    "minFreeMemoryMiB": 768,
    "thermalPauseAt": "SEVERE"
  }
}
```

13.4 First-generation and login storms

- Treat random-bot creation/generation as a distinct job. It may create many accounts/characters and perform substantial SQL work.
- Generate in bounded batches and persist the last completed identity/range so a kill resumes without duplicates.
- Do not start all target bots at once. Ramp logins in small batches and observe world tick and free memory.
- Expose “bots available” separately from “bots online.” Character generation success is not proof of runtime capacity.
- Provide Cancel safely between batches; cancel must not leave an ambiguous schema/revision state.
- On downgrade to a lower tier, log bots out gradually. Do not delete generated characters automatically.

13.5 Admission controller

A lightweight controller should prevent the bot manager from chasing a configured count when the device is already overloaded. It may request a lower online target through supported configuration/commands, but it must not patch world memory or kill bot sessions arbitrarily.

Conceptual bot admission loop

```
every 10 seconds:
    observe p99WorldUpdateMs, freeMemoryMiB, processRss, thermalStatus
    if thermal >= SEVERE or freeMemoryMiB < floor or p99WorldUpdateMs > hardLimit:
        reduce requested online bots by step; record reason
    else if healthy for 5 minutes and online < selected target:
        increase requested online bots by small step
    enforce min/max and cooldown
    never change persistent user-selected profile without showing adaptation state
```

13.6 Bot-specific regression scenarios

ID	Scenario	Pass condition
BOT-T01	25 bots idle across world for 2 hours	No OOM; stable world tick; database growth recorded.
BOT-T02	Player forms party with 4 bots and completes ordinary combat/loot loop	No stuck neutral target loop; loot latency acceptable.
BOT-T03	Water/shore pathing and follow behavior	No repeated line-of-sight/path oscillation; missing mmaps detected.
BOT-T04	Logout/login player while random bots remain	Player returns; world and bot state stable.
BOT-T05	Force-stop during bot generation	Resume without duplicate account/character range.
BOT-T06	Raise 25→50 and lower 50→25 during session	Gradual convergence; no login storm or database error.
BOT-T07	Database/storage approaches configured floor	Admission stops growth and warns; no disk-full transaction.
BOT-T08	Thermal severe event	Bot ramp pauses/reduces; client remains controllable.
BOT-T09	AH automation enabled on qualified profile	Separate CPU/SQL budget and restore test pass.

13.7 Known anecdotal issue themes

Community reports around Vanilla playerbots mention target-selection loops, inefficient melee behavior, pulling and looting slowdowns; SPP users also report pathing or line-of-sight issues in difficult terrain/water. These are not universal facts, but they should become reproducible scripted tests before increasing bot tiers. [S25–S27]

14. Local account creation and first-run experience

14.1 Provision accounts through the core

Use CMaNGOS-supported console commands such as account create and account set gmlevel through the local control bridge. Do not directly manufacture SRP/verifier fields or depend on a guessed database password algorithm. This keeps account behavior aligned with the pinned core and produces meaningful validation errors. Community Android server instructions also use server console account commands, reinforcing this route. [S08, S11]

14.2 Account workflow

1. Require the database and world control channel to be ready; the game client need not be running.
2. Validate username and password locally using limits derived from the pinned core, but let the core remain authoritative.
3. Send the password through an in-memory protected path; avoid command-line arguments, environment variables and logs.
4. Execute account creation and parse a structured result from the Android control module.
5. Optionally set expansion/access level consistent with Vanilla and a non-GM default.
6. Offer a separate, explicit “local administrator/GM” toggle with warning; do not make every account GM by default.
7. Return account ID and friendly status; zeroize password buffers as practical and do not persist the password unless the user opts into Android credential storage.

14.3 Account input rules

Rule	Behavior
Username	Normalize according to core behavior, show final form, reject whitespace/control characters and overlength before submission.
Password	Do not trim silently; warn about legacy client/core constraints; never include in exception messages.
Duplicate	Return ACCOUNT_EXISTS and offer sign-in/change-password path.
GM level	Default 0. Higher levels require explicit advanced action and an audit log without password.
Delete	Require typed confirmation and a current consistent backup; explain character/world effects.
Password change	Route to supported core command; invalidate any optional stored credential.

14.4 First-run wizard stages

Stage	User action	Automated checks	Completion evidence
F01 Welcome	Acknowledge user-owned files/local realm	Device ABI, RAM, storage, Vulkan/OpenGL, API/page size	Capability report saved.
F02 Import	Select WoW folder	Build 5875, source layout, storage calculation	Managed client manifest.
F03 Server data	Start/pause extraction	Thermal/storage/checkpoints	Data manifest.
F04 Realm initialization	Choose realm name and base profile	DB init/migrations/core data validation	Stopped-ready stack.
F05 Account	Create local account	Core command result	Account ID.
F06 Controls	Choose touch/gamepad/keyboard profile	Input self-test	Profile saved.
F07 First start	Press Start	Full preflight and safe graphics profile	Character-select reached.
F08 Bots	Optional enable after first play	Zero-bot baseline exists	Selected profile generated/ramped.

14.5 Preflight before every start

ID	Required assertion
P-01	No import/extraction/migration job owns mutable runtime paths

ID	Required assertion
P-02	Managed client manifest valid and build 5875
P-03	Data manifest compatible with core and required outputs present
P-04	Database manifest/revision compatible; recovery not pending
P-05	Enough free storage for database log growth and session cache
P-06	Ports/socket paths available; no stale component session owns them
P-07	Client backend capability probe passes for ABI, 32-bit and graphics
P-08	Foreground notification permission/service start is available
P-09	Bot profile is within device-qualified bounds
P-10	Last session journal is STOPPED or recovery completed

14.6 Progress messaging

Use specific, bounded stages rather than an indeterminate “Starting server.” A normal zero-bot start should show: Checking files → Starting database → Checking schemas → Starting login service → Loading world → Applying bot profile → Starting Windows runtime → Opening client. Each stage has a timeout, a retry policy and a diagnostic code. Do not invent percent complete for a core startup phase unless progress is based on real milestones.

14.7 Safe first-start defaults

Setting	Default
Bots	Off for the first successful character login; offer 25 afterward.
AH automation	Off.
Resolution	1280×720 virtual desktop or the nearest aspect-fit below device native.
Frame cap	30 FPS.
Renderer	WineD3D/safe OpenGL path on emulator until Vulkan path is proven.
Audio	Enabled with moderate buffer; one-click disable for diagnostics.
Network	IPv4 loopback only.
Graphics settings	Low/medium baseline, reduced view distance, shadows off or low.
Account GM	Level 0.
Logs	Info with bounded persisted size; native crash/tick metrics enabled.

15. x86-64 Windows-client runtime

15.1 The purpose of the x86 milestone

The x86/x86-64 Android emulator is not merely a convenience. It removes ARM CPU translation from the first integrated proof, allowing the agent to isolate Wine, graphics, Android packaging, client compatibility and local networking. It must not, however, become a false promise that the ARM build is an ABI rebuild. The client runtime contract exists specifically so the later ARM backend can replace the execution engine without rewriting import, server, accounts or lifecycle.

15.2 Why stock Winlator is not the x86 implementation

Winlator is an Android application built around Wine plus Box86/Box64 for running Windows x86/x86-64 applications. Its upstream Android packaging is ARM64-oriented and its architecture assumes a translation/container stack suited to ARM. Reusing display, input, container or configuration ideas may be valuable, but the x86 emulator should run an x86 Wine build directly. Do not spend the first milestone porting Box64 to a platform where it is unnecessary. [S09]

15.3 ClientRuntime interface

Required backend boundary

```
interface ClientRuntime {
  suspend fun probe(device: DeviceCaps, client: ClientManifest): ClientCaps
  suspend fun preparePrefix(request: PrefixRequest): PrefixResult
  suspend fun launch(request: LaunchRequest): ClientSession
  suspend fun requestClose(sessionId: UUID): CloseResult
  suspend fun forceStop(sessionId: UUID)
  fun observe(sessionId: UUID): Flow<ClientEvent>
  suspend fun collectDiagnostics(sessionId: UUID): ClientDiagnostics
}

data class ClientCaps(
  val supported: Boolean,
  val windowsArch: Set<WindowsArch>,
  val renderers: List<RendererCaps>,
  val audioBackends: List<AudioCaps>,
  val reasons: List<CapabilityFailure>,
  val runtimeBuildId: String
)
```

15.4 x86 execution combinations to test

Android image	Native app ABI	Wine strategy	Expected value
x86 API 28	x86	32-bit x86 Wine	Simplest architecture match; useful legacy research lane.
x86_64 API 28 with x86 ABI	x86_64 supervisor + x86 client service/runtime	32-bit x86 Wine	Tests mixed-ABI packaging/process assumptions.
x86_64 modern with 32-bit userspace	x86_64	32-bit Wine or multilib	Closer to production policy while retaining direct execution.
x86_64 modern, 64-bit-only	x86_64	64-bit Wine WoW64 running 32-bit WoW.exe	Most future-facing x86 lane; may expose Wine WoW64 issues.
x86_64 16 KB AVD	x86_64	Whichever modern lane passes	Native-library/page alignment compliance, not performance.

15.5 Wine prefix ownership

- Create a dedicated prefix for this product and client identity. Never share a user-selectable arbitrary Wine prefix.
- Record Wine build ID, prefix schema, Windows architecture, renderer and installed runtime components in prefix-manifest.json.
- Prepare the prefix before the first server start where possible, so Wine initialization errors are not confused with realm errors.
- Treat registry and prefix state as mutable user data; back it up only as an optional diagnostic/settings artifact, not the authoritative world state.
- When Wine or architecture changes incompatibly, stage a new prefix and preserve the old one until the client reaches character-select.

- Keep WoW files outside the prefix if the runtime supports a stable mapped drive; otherwise use a deterministic app-private mapping.

15.6 Launch request

Illustrative normalized launch request

```
LaunchRequest(
  executable = managedClient.resolve("WoW.exe"),
  workingDirectory = managedClient.root,
  arguments = emptyList(),
  environment = mapOf(
    "WINEPREFIX" to prefix.path,
    "WINEDEBUG" to selectedLogChannels,
    "PULSE_LATENCY_MSEC" to audioProfile.latencyHint
  ),
  virtualDesktop = Size(1280, 720),
  renderer = Renderer.WINED3D,
  frameCap = 30,
  networkMode = LOOPBACK_ONLY,
  inputProfileId = "touch-standard-v1"
)
```

15.7 Launch and readiness sequence

1. Probe runtime/code artifacts, Android ABI and 32-bit/WoW64 capability.
2. Verify no stale wineserver/client process from another session; terminate only processes bearing the app session token.
3. Prepare or migrate prefix and run a lightweight Wine self-test.
4. Ensure managed realmlist points to 127.0.0.1 and world readiness has been signaled.
5. Create virtual desktop/display surface and establish input bridge before WoW.exe starts.
6. Launch WoW.exe with cwd set to managed client root and capture stdout/stderr/Wine debug channels.
7. Detect the game top-level window/surface, transition CLIENT_STARTING→RUNNING and hide setup progress behind the game surface.
8. If login UI appears but realm login fails, classify network/protocol separately from renderer/client startup.

15.8 WoW client failure classification

Code	Observed symptom	Likely layer	Next diagnostic
CLI-PE	WoW.exe cannot be loaded/bad format	Wrong Wine architecture or corrupt client	PE machine/build and Wine loader trace.
CLI-DLL	Missing DLL/import	Wine build/prefix/runtime	Wine loader channels; compare clean prefix.
CLI-BLACK	Window opens black	Renderer/surface/shader	Switch WineD3D/DXVK; disable overlay; capture GPU logs.
CLI-LOGIN-DROP	Credentials accepted then disconnected	Realm build/address/world readiness	realmd/world logs, realm row, build 5875, ports.
CLI-REALMLIST	Repeated realm list / cannot enter	Realm address/DB row/client cache	RealmList table, realmlist.wtf, guest loopback, WDB clear test.
CLI-AUDIO	Hang/crackle when audio initializes	Audio backend/buffer	Launch with audio off; backend trace.
CLI-INPUT	Game visible but no/corrupt input	Focus/pointer/input bridge	Overlay off, physical keyboard test, stuck-key reset.
CLI-CRASH-ZONE	Crash entering world/zone	Graphics/data/addon/client corruption	Safe graphics, addons off, WDB clear, exact zone.
CLI-FPS	Very low FPS despite idle server	Renderer/virtual desktop/host GPU	Frame timing and renderer A/B.

15.9 Client safe mode

Implement one deterministic safe-mode launch that is more useful than a generic “try again.” It should select WineD3D, 1280×720 or lower, 30 FPS cap, low game settings, audio off, overlays minimal, app-installed addons disabled, WDB/cache cleared only after confirmation, and a clean temporary prefix if the normal prefix fails. The diagnostic bundle must record every safe-mode deviation.

15.10 No launcher or anti-cheat dependencies

The intended 1.12.1 client launches WoW.exe directly and connects to a local realm. Community reports indicate that older clients without modern launchers or anti-cheat are the practical Winlator use case, while some users see login/disconnect failures depending on server/client setup. Treat direct WoW.exe launch as a validation requirement and reject imported setups that require an unknown proprietary launcher or injected module. [S21, S22]

16. Graphics, audio, input and mobile UX

16.1 Renderer selection policy

Backend	Use first on	Strength	Risk/fallback
WineD3D → OpenGL	x86 emulator and generic safe mode	Broad compatibility, fewer Vulkan dependencies	Lower performance/driver translation; reduce resolution.
DXVK → Vulkan	Qualified Vulkan emulator and ARM devices	Potentially much better Direct3D performance	Vulkan/driver-specific black screens or shader issues; fall back.
Turnip + DXVK	Qualified Adreno ARM devices	Common high-performance Winlator path	Not applicable to Mali; pin Mesa/Turnip build and device profile.
VirGL/Zink path	Some emulator/Mali fallback cases	May bridge available host/guest graphics	Performance and correctness vary sharply.
Software renderer	Diagnostics only	Separates GPU from client failure	Not playable; hard time limit and warning.

16.2 Capability-based renderer decision

Renderer policy concept

```
if debugForceRenderer is set:
    use it and mark unsupported override
else if backend == X86_DIRECT and wineD3D.selfTestPasses:
    default = WINED3D
else if gpu.vendor == QUALCOMM and turnipBuild.supports(gpu) and dxvk.selfTestPasses:
    default = DXVK_TURNIP
else if systemVulkan.level >= required && dxvk.selfTestPasses:
    default = DXVK_SYSTEM
else:
    default = WINED3D

on two consecutive renderer-start failures:
    offer Safe Mode; do not loop renderer switches automatically
```

16.3 Resolution and frame pacing

Profile	Virtual desktop	Frame cap	Use
Battery/compatibility	960×540 or 1024×576	30 FPS	Low-end/thermal fallback.
Balanced default	1280×720	30 or 45 FPS	Initial mobile target and emulator baseline.
Quality	1600×900	45/60 FPS	Qualified high-end devices only.
Native-like	Device-aspect fitted, up to 1920×1080	60 FPS	Advanced; never default.

Avoid rendering at a modern phone's full native resolution by default. Scale the virtual desktop into a letterboxed or aspect-fit surface and transform input coordinates through one tested matrix. Log render resolution, surface resolution and scale factor separately.

16.4 Frame-time acceptance

Target	Frame budget	Acceptance
30 FPS	33.3 ms	95th percentile ≤ 33.3 ms in ordinary questing; 99th percentile reported.
45 FPS	22.2 ms	Optional profile only when reference scene passes without severe thermal state.
60 FPS	16.7 ms	Qualified devices; not a product-wide guarantee.
Input latency	Planning target ≤ 100 ms touch-to-visible action	Measure with high-speed video/instrumentation; compare overlay off/on.

16.5 Audio

- Provide a complete “Audio off” diagnostic path before tuning latency.
- Prefer one stable Android audio bridge/backend and expose only buffer presets, not arbitrary environment variables.
- Record underruns, sample rate, channel count and selected buffer in diagnostics.
- Pause/mute on focus loss according to user setting; do not stop the realm when another app briefly takes audio focus.
- Bluetooth adds latency and route changes; test wired, speaker and Bluetooth separately.
- If audio initialization blocks client launch, time it independently and allow one automatic retry with audio disabled.

16.6 Input architecture

Vanilla 1.12.1 has no native modern gamepad UI. The Android app must synthesize keyboard and relative/absolute mouse events for Wine, while an optional 1.12-compatible addon can improve action layout. ShaguController is an example of a controller-oriented Vanilla 1.12 addon; it should be treated as optional open-source UI assistance, not as a replacement for the Android input bridge. [S28]

Input source	Translation
Left virtual stick	W/S forward/back and A/D strafe, with dead zone and hysteresis.
Right virtual stick/drag region	Relative mouse movement while camera-look button/state is active.
Tap on world/UI	Absolute pointer move + left click; coordinate transform accounts for letterboxing.
Long press / secondary button	Right click for interaction/context.
Action buttons	Configurable keyboard bindings 1–=, modifiers and selected keys.
Gamepad sticks/buttons	Same logical action map; support remapping and per-device dead zones.
Physical keyboard/mouse	Direct passthrough with pointer capture; overlay may auto-minimize.
Android IME	Open explicit chat text field, convert committed text and Enter/Escape to game input.

16.7 Default touch layout

Control	Default position/behavior	Notes
Movement stick	Lower-left; floating-center option	Four-way with diagonal; visual key-state feedback.
Camera region	Right half drag	Relative mode; sensitivity separate from cursor.
Interact/target	Right-side primary buttons	Map to target/interact bindings selected in game.
Action bar	8 visible + modifier page for 8 more	Opacity/size adjustable; avoid covering chat/quest text.
Menu cluster	Bags, character, map, quest log, spellbook	Collapsible radial or strip.
Chat	Dedicated button opens IME bridge	Show send/cancel; prevent stuck movement while typing.
Mouse mode toggle	Cursor versus camera	Persistent visual state.
Overlay hide	Gesture or button	Required for screenshots/UI interaction and physical peripherals.

> **Mobile usability is a release gate**
>
> Community users report that touch overlays can occupy roughly half the screen and that chat and precise target selection are recurring problems. A working login with unusable controls is not a successful port. Test touch-only questing and chat end to end. [S23, S24]

16.8 Stuck-input prevention

- Maintain a logical pressed-key/button set per input source and synthesize releases on activity pause, focus loss, overlay restart, client exit and profile switch.

- Do not map a drag that begins on a UI button into camera motion after it leaves the button unless explicitly designed.
- When IME opens, release movement keys and suspend camera capture.
- On Android gesture navigation conflicts, offer inset calibration and a compact layout.
- Persist profiles, but reset to a known layout when screen aspect/resolution changes beyond a tested threshold.

16.9 Addon strategy

Addon class	Policy
App-vetted 1.12 control addon	Optional install; exact version/license pinned; can be disabled in safe mode.
User addons from source client	Copy by default with warning; detect obvious non-1.12 TOC/interface mismatch where possible.
Modern ConsolePort/retail addons	Do not describe as Vanilla-compatible without an actual 1.12 port; many target later clients.
Addon failure diagnosis	Launch with Interface/AddOns temporarily disabled, preserve files, compare.

16.10 UX acceptance scenarios

ID	Touch-only task	Pass condition
UX-T01	Create character and enter world	No physical keyboard/mouse required.
UX-T02	Move, turn camera and stop reliably	No stuck movement across focus/IME changes.
UX-T03	Target an NPC, accept and turn in a quest	Target/click precision sufficient at default scale.
UX-T04	Fight three mobs using at least six abilities	Buttons readable; camera and action use do not conflict.
UX-T05	Loot, open bags, equip item	Pointer mode reachable and accurate.
UX-T06	Type a chat sentence with punctuation	IME commits correctly; movement released; message sends once.
UX-T07	Invite/control an alt bot	Required slash/whisper/command workflow achievable.
UX-T08	Rotate device or resume from temporary overlay/activity interruption	Surface recovers or orientation is deliberately locked with no input corruption.

17. Networking and realm advertisement

17.1 Local-only network model

For the MVP, every service exists inside the Android guest/application and the client connects to loopback. Use IPv4 127.0.0.1 consistently until IPv6 is explicitly tested. Database traffic should use a Unix socket. realmd conventionally listens on TCP 3724, world on TCP 8085, and the realm database advertises an address that the Wine client can reach. [S06]

Local-only endpoint baseline

```
Client realmlist:  set realmlist 127.0.0.1
Realm DB address: 127.0.0.1
Realm port:       8085
realmd bind:      127.0.0.1:3724
mangosd bind:     127.0.0.1:8085
MariaDB:          <app-private Unix socket>, no TCP
```

17.2 Realm row ownership

- The product owns the local realm row and repairs it from typed settings during migration/preflight.
- Use a stable realm ID (for example 1) inside the single-realm product and verify no duplicate rows.
- Address must be 127.0.0.1 for integrated guest mode. Do not write 10.0.2.2 unless intentionally connecting to a Windows-host diagnostic server.
- The advertised build range/version must accept build 5875 according to the pinned core.
- Realm name is user-editable within validated encoding/length; changing it does not create a new world database.
- Persist a diagnostic snapshot of the effective row without database secrets.

17.3 Readiness probes

Endpoint	Probe	Success
MariaDB socket	Connect with core service account; SELECT 1 and revision query	Authentication, schema and revision all pass.
127.0.0.1:3724	TCP connect plus realmd structured ready event	Both pass for same session ID.
127.0.0.1:8085	TCP connect plus world structured ready event	Both pass; server has loaded data/scripts.
Client realm login	Instrumented/manual login	Authentication → realm list → world connection.

17.4 Emulator host bridge

The Android Emulator reserves addresses in a virtual network. 10.0.2.2 aliases the development host loopback, whereas 127.0.0.1 remains the emulator. Use the host bridge in two temporary tests only: client-in-guest against a known-good host realm, and Android server against a host diagnostic client if protocol tooling requires it. A release acceptance run must have network access disabled and still pass. [S19]

17.5 LAN mode as a future feature

Concern	Requirement before enabling
Bind address	Explicit user action; selected interface; persistent visible warning.
Android local-network permissions	Re-evaluate current Android restrictions/permissions at implementation time, including newer local-network controls.
Firewall/router	No automatic UPnP or Internet exposure.
Admin/database	Never expose DB or control socket. Only auth/world endpoints as designed.
Realm address	Advertise device LAN IP and handle changes; local client may need separate local address behavior.
Authentication	Strong account passwords; rate limiting/logging; no default GM credentials.
Support boundary	Separate test matrix and security review; not implied by local-only success.

17.6 Network failure playbook

Symptom	Check order
Client cannot connect at all	realmlist file → 3724 listener → realmd ready → loopback namespace → firewall/VPN interference.
Login succeeds, realm list loops	realm row address/port/build → world listener → realm/world DB compatibility → client WDB cache.
Realm shown offline	realmd world-status update → realm row → world startup completion → time/session mismatch.
Disconnect on character entry	world log exact error → build 5875 → data maps/DBC → account expansion/access → client addon/cache.
Works against 10.0.2.2 host but not integrated	Guest server bind/readiness; do not change client runtime first.

18. Process lifecycle, foreground execution and shutdown

18.1 Startup dependency graph

No stage launches merely because a previous process has a PID

```
preflight
  └─ database.start
      └─ database.ready + schemas compatible
          └─ realmd.start
              └─ realmd.ready (3724)
                  └─ mangosd.start
                      └─ world.ready (8085 + data/scripts loaded)
                          └─ clientRuntime.launch
                              └─ client.windowReady
                                  └─ RUNNING
```

18.2 Timeouts and retries

Stage	Initial planning timeout	Retry policy
Database start/recovery	30–120 s depending on dirty state	One controlled retry only after classifying recoverable startup.
realmd ready	30 s	No blind repeated spawn; stop existing session and report logs.
World ready, 0 bots	120 s cold / 60 s warm planning values	No auto-retry on DB/data/schema errors.
World ready with bots	Up to 300 s measured	Show bot/generation state; distinguish slow from stalled.
Wine prefix self-test	60 s	One clean-prefix diagnostic retry.
Client window	90 s	Offer safe mode; do not restart whole realm automatically.
Graceful world stop	60 s normal, configurable for save	Escalate after log/heartbeat check.
Database clean stop	30 s	If forced, mark dirty and require recovery next start.

All durations above are planning estimates. Persist observed p50/p95 by device/profile and use evidence to refine. The UI should show elapsed time and the last concrete milestone rather than an invented percentage.

18.3 Graceful shutdown sequence

1. Disable new Start/reconfiguration operations and mark STOPPING in the durable journal.
2. Ask the client to close cleanly; allow a short user-facing grace period. The world remains available during this step.
3. Send a core-supported world save command and wait for acknowledgment or a bounded save-complete signal.
4. Send server shutdown with zero/short delay through the control channel; continue collecting logs.
5. Wait for mangosd to exit. If unresponsive, send TERM-equivalent through the managed native service, then KILL only after timeout and mark dirty.
6. Stop realmd and wait for exit.
7. Flush/checkpoint and stop MariaDB. Verify socket removal/process exit.
8. Fsync journal, mark STOPPED with clean=true only if every durability step succeeded, then remove the foreground notification.

18.4 App/UI lifecycle behavior

Event	Required behavior
Activity backgrounded briefly	Realm/client continue under foreground service; notification returns to game.
Screen off	Default: keep session alive for a short configurable period or pause client rendering; warn about battery. Realm policy explicit.
Task swiped away	Do not assume stop; foreground runtime continues unless product explicitly maps swipe to stop and can complete it.
User presses notification Stop	Save and stop without opening activity.
Android force-stop	No callback guarantee. Next launch detects dirty journal and recovers.

Event	Required behavior
OS kills client process only	Supervisor offers Relaunch Client; realm remains for bounded window.
OS kills supervisor	Other processes must not become uncontrolled daemons; component services detect binder/session loss and apply safe policy.
Configuration change	UI recreates; runtime state remains in service.
Low-memory callback	Reduce caches/bot admission where safe; never claim it prevents process death.

18.5 Durable supervisor journal

Journal writes use atomic temp+fsync+rename or a small transactional database

```
{
  "schema": 2,
  "sessionId": "uuid",
  "phase": "WORLD_STARTING",
  "requestedProfile": "mobile-low-v1",
  "clean": false,
  "components": {
    "database": {"state":"READY","instanceToken":"...","startedAt":"..."},
    "realm": {"state":"READY","instanceToken":"..."},
    "world": {"state":"STARTING","instanceToken":"..."},
    "client": {"state":"STOPPED"}
  },
  "lastDurableAction": "database-schema-compatible",
  "lastError": null,
  "updatedAtWall": "...",
  "updatedAtElapsedMs": 123456
}
```

18.6 Orphan detection

PIDs can be reused, so do not kill a process based solely on a stale PID file. Assign an unguessable instance token/session ID to every component through an inherited descriptor, Binder registration or environment plus control handshake. At recovery, identify package-owned processes and confirm their token before signaling. Any unrecognized listener on required ports is reported as PORT_IN_USE, not killed.

18.7 User stop and Android policy

Android gives users direct control over active foreground apps. Respect user stop/force-stop and never schedule a hidden job to resurrect the realm. The next explicit Start may recover state, but the app must explain that the previous session ended unexpectedly. [S16, S18]

19. Diagnostics, support bundles and recovery

19.1 Structured logging

Field	Required
wallTime + elapsedTime	Both; elapsed time supports ordering across wall-clock changes.
sessionId	Null outside session; stable across all components within a run.
component	ui, supervisor, database, realm, world, wine, client, input, worker.
severity	TRACE/DEBUG/INFO/WARN/ERROR/FATAL with release-level filtering.
eventCode	Stable machine-readable code for major transitions/failures.
message	Human-readable, no secrets.
attributes	Typed bounded values; paths redacted or made relative.
nativeBuildId	For crash/symbol matching.

19.2 Log retention

- Keep an in-memory ring for live UI and bounded per-component files on disk.
- Rotate by size and session; a planning limit is 10–25 MiB per major component and 100 MiB total, then measure.
- Preserve the latest failed startup and dirty-shutdown session even when normal logs rotate.
- Wine debug channels can explode in volume; default to errors/fixmes needed for diagnosis and enable verbose channels for one bounded reproduction.
- Database general query logging stays off by default because of performance, volume and credential/content exposure.

19.3 Support bundle contents

Included	Redaction/limit
App version, manifests, commits, licenses	No redaction needed.
Device/AVD capability report	Model/build/API/ABI/GPU/driver/RAM/page size; user consent.
Supervisor journal and event log	Remove source document URI and absolute personal paths.
Last N component logs	Redact passwords, DB credentials, account email-like strings and tokens.
Effective server configuration	Replace DB password and private paths with placeholders.
Database revision/status	No row dumps or character/account data by default.
Client manifest	Hashes/relative paths; do not include client files.
Graphics/audio/input profile	Include exact backend/build and overlay calibration.
Crash tombstone/backtrace	Symbolized where possible; scan memory strings for secrets before export.
Performance snapshot	RSS, CPU, world tick, frame times, thermal and free storage.

19.4 Redaction tests

Maintain synthetic secrets in integration tests and fail the export if any appear. Test uppercase/lowercase username, password, database password, source folder name, document URI, Android account name and IP addresses outside allowed loopback. Redaction must operate on structured fields first and regex only as a defense-in-depth layer.

19.5 Repair actions

Action	Scope	Safety
Recheck files	Managed client and data manifests	Read-only; deep hash optional.
Regenerate realmlist/config	App-owned text config	Atomic and reversible.
Reset Wine prefix	Wine registry/runtime state	Preserve client/WTF/addons and world DB; confirm.
Clear client cache	WDB/cache only	Show exact directories and side effects.
Rebuild vmaps/mmaps	Versioned server data	World stopped; staged publish.

Action	Scope	Safety
Repair realm row	One product-owned DB row	Snapshot row; typed expected state.
Run database recovery	MariaDB-supported recovery	Preserve evidence; no destructive automatic repair.
Restore backup	World/account/database state	Show timestamp/core version; automatic pre-restore safety copy where space permits.
Reset realm	Database/data/config	Destructive; typed confirmation; client copy remains.
Full reset	All app-managed runtime data	Destructive; user source unaffected.

19.6 Recovery decision tree

Dirty-session recovery policy

```
on launch:
  if journal.clean == true and all components absent:
      normal STOPPED
  else:
      stop/identify any package-owned stale components
      inspect database error log and recovery requirement
      if DB starts and consistency/revision checks pass:
          stop DB cleanly; mark RECOVERED; offer Start
      else if valid compatible backup exists:
          offer Restore with evidence preserved
      else:
          block world start; offer diagnostic export and explicit reset/advanced recovery

never auto-loop more than one recovery attempt per app launch
```

19.7 Developer diagnostic panel

- Component process/service state, instance tokens and last heartbeat.
- Open socket table and readiness probe results.
- Effective ABI, Wine architecture, renderer and native build IDs.
- Database revisions and migration ledger status.
- World tick p50/p95/p99, online bots, client FPS/frame-time percentiles, RSS/PSS and thermal status.
- One-click capture markers for “problem starts now” and “problem ends now.”
- Fault injection controls in debug builds only: kill each component, fill staging quota, revoke source permission and simulate port conflict.

20. x86-64 Android-emulator implementation sequence

20.1 Reference environment

Record the Windows host CPU, virtualization backend, Android Emulator version, system-image ID, API, ABI list, GPU mode, RAM, data partition size and page size in source control. An “Android x86-64 emulator” is not sufficiently specific for reproducibility.

AVD	Suggested characteristics	Purpose
AVD-Legacy-x86	API 28; x86 or x86_64 with x86 support; Google APIs optional; 6–8 GiB RAM; 24+ GiB data	Fast research lane for direct 32-bit Wine and unpack/exec.
AVD-Modern-x86_64	Current production target-compatible image; x86_64; hardware graphics; 8 GiB RAM	Modern packaging, FGS, scoped-storage and lifecycle.
AVD-16K-x86_64	Android 15+ experimental 16 KB x86_64 image	Native library and mmap/page-size compliance. [S14]
AVD-LowMem	Same modern image; 4 GiB RAM or constrained profile	Memory pressure and admission behavior.

20.2 Gate sequence

Gate	Focus	Exit criterion
X0	Capability capture	App displays API, ABI lists, page size, graphics and allocatable storage; support bundle matches adb.
X1	Native packaging spike	Package-native hello/service loads and exits on legacy and modern AVD; deliberate crash isolated.
X2	Wine self-test	x86 Wine backend initializes prefix and displays a non-proprietary test window inside app surface.
X3	WoW client render	Imported build-5875 WoW.exe reaches login screen with no server.
X4	Native MariaDB	Initialize, query, clean stop, dirty kill and recovery on x86_64 Android.
X5	Native realmd	Connects to DB and listens on 127.0.0.1:3724; structured ready event.
X6	Native world, no bots	Loads pinned DB and client data; listens on 8085; clean save/stop.
X7	Manual integrated login	Client authenticates, creates/loads character, plays 30 minutes, restarts with state.
X8	Supervisor integration	One Start/Stop action with explicit stages and 20 clean cycles.
X9	Importer/extractor	Fresh app completes resumable setup from SAF; 10 kill/resume tests.
X10	Bots low	25-bot profile and 2-hour soak pass.
X11	Touch UX	UX-T01–T08 pass without physical peripherals.
X12	Modern/16 KB	Same runtime passes current target and 16 KB tests without legacy execution assumptions.

20.3 X2–X3: prove the client before the realm

1. Select an AVD that advertises the ABI needed by the direct Wine payload; verify with getprop, not the AVD label.
2. Package a minimal, legally redistributable Windows test executable and prove Wine display/input/audio lifecycle.
3. Import the user-owned client through a temporary debug path and validate PE x86/build 5875.
4. Launch WoW.exe with WineD3D, 1280×720 and audio off. Reaching a stable login UI proves client/render startup independent of server behavior.
5. Add keyboard/mouse input, then Android touch translation. Keep the realm absent so login errors are expected and non-confounding.
6. Capture a clean prefix and launch profile as the x86 client golden baseline.

20.4 X4–X6: prove the native server separately

1. Run MariaDB unit/startup tests in :database with an app-private socket and no TCP.
2. Import only the pinned schemas; test revision mismatch intentionally.
3. Start realmd with a known realm row; probe 3724 and stop cleanly 20 times.
4. Prepare client-derived data using a known-good development copy; start mangosd with playerbots disabled.
5. Verify world-ready, execute account create, save and shutdown through the control socket.
6. Kill mangosd and MariaDB at controlled points and verify the recovery classifications before adding the client.

20.5 X7: integration checklist

Step	Expected evidence
Write managed realmlist	File content and locale path logged by relative path/hash.
Start database	Socket query and schema digest.
Start realmd	Structured ready + 3724 listener.
Start world	Structured ready + 8085 listener + data manifest ID.
Create account	Structured ACCOUNT_CREATED or ACCOUNT_EXISTS.
Launch client	Wine build/renderer/window-ready event.
Authenticate	realmd account/build acceptance; no password in logs.
Select realm	Realm address resolves to guest loopback; world connection logged.
Create character	Character row and in-game entry.
Play 30 minutes	Frame/tick/memory series and no fatal logs.
Save/stop/restart	Clean journal; same character position/inventory/quest state.

20.6 Host-side diagnostic bridge

> **Use 10.0.2.2 only to isolate layers**
>
> If the Android client cannot log into the Android server, connect it temporarily to a known-good CMaNGOS instance on the Windows host at 10.0.2.2. If that works, focus on the Android server/network plane. Then test an external diagnostic client against the Android realm only if a controlled port-forwarding method is available. Remove every host dependency before X7 passes. [S19]

20.7 Emulator performance caveat

Use emulator measurements for regression, not as direct phone performance predictions. Host GPU translation, Windows hypervisor configuration, CPU scheduling and AVD RAM alter results. Record relative changes under a fixed AVD and move performance qualification to at least one Adreno and one Mali physical device in the ARM phase.

20.8 x86 failure triage order

1. Confirm exact AVD ABI lists and page size.
2. Confirm the package contains the correct native libraries and no host-architecture contamination.
3. Run the smallest runtime self-test in the same service/process model.
4. Prove WoW login screen without server.
5. Prove database/realm/world with client absent.
6. Verify loopback endpoints and realm row.
7. Integrate with bots off, addons off, safe renderer and audio off.
8. Only then add bots, addons, DXVK and touch-overlay complexity.

21. ARM64 transition strategy

21.1 What should remain unchanged

Layer	ARM transition expectation
Importer/managed client	Unchanged Kotlin logic; same build-5875 manifest.
Database schema/data	Unchanged; native MariaDB binary changes ABI only.
CMaNGOS/playerbots	Same commits, patches, configs and control protocol; rebuild arm64-v8a.
Accounts/bots/backup	Unchanged service contracts and UI.
Supervisor/lifecycle	Unchanged state machine; timeouts may be profile-specific.
ClientRuntime	Replace x86DirectWine with armTranslatedWine.
Renderer/input	Same logical profiles; backend/device-specific implementation/tuning.
Performance qualification	New baselines and limits required.

21.2 ARM client branches

Branch	Precondition	Stack	Risk
ARM-A	Device supports required 32-bit execution environment	ARM64 app/server + Box86/32-bit Wine client path	Device/vendor 32-bit support is shrinking; packaging complexity.
ARM-B	64-bit-only Android supported	ARM64 app/server + Box64 + 64-bit Wine WoW64 running 32-bit WoW.exe	Wine/Box WoW64 edge cases; must qualify per build.
ARM-C	Unsupported translator/GPU combination	No client launch; server-only diagnostic not exposed as product mode	Fail capability preflight with exact reason.

21.3 Device capability probe

ARM capability model

```
DeviceCaps {
  androidApi
  androidAbis32 / androidAbis64
  totalRamMiB / memoryClassMiB
  pageSize
  gpuVendor / gpuRenderer / driverVersion
  glVersion / vulkanVersion / extensions
  storageAllocatableBytes
  thermalApiAvailable
}

ArmRuntimeProbe {
  box64SelfTest
  box86SelfTestIfPackaged
  wine32SelfTest
  wineWow64SelfTest
  dxvkSelfTest
  wineD3dSelfTest
}

select a backend only from successful self-tests; never infer from model name alone
```

21.4 Translation-runtime requirements

- Pin Box64/Box86 and Wine builds together. A translator update is not independent from Wine and graphics translation.
- Package CPU-translation code as immutable APK native code under the production execution model; do not download executable updates into writable storage.
- Keep translator caches in app-private mutable storage with size quotas and clear-cache repair.
- Expose safe environment tunables only through signed device profiles. Do not present hundreds of Box/Wine variables to normal users.
- Log translator build ID, selected mode, cache size and notable fallback. Avoid verbose instruction tracing outside bounded diagnostics.

- Test 32-bit Windows thread-local storage, exception handling, memory mapping, large-address behavior and self-modifying/JIT code under 4 KB and 16 KB pages.

21.5 GPU family strategy

GPU/device family	Primary candidate	Fallback	Qualification
Qualcomm Adreno	Pinned Turnip Mesa + DXVK	System Vulkan DXVK, then WineD3D	At least two GPU generations and Android versions.
ARM Mali	System Vulkan DXVK or tested Zink/VirGL path	WineD3D	At least one modern and one mid-range device; watch driver bugs.
PowerVR/other	System Vulkan if self-test passes	WineD3D	Best-effort/unsupported until device passes complete gates.
No usable Vulkan	WineD3D/OpenGL	Lower resolution/software diagnostic	30 FPS profile may be unattainable; capability warning.

21.6 ARM performance sequence

1. Build and pass native arm64 MariaDB/realmd/mangosd tests with the client absent.
2. Run translator/Wine self-test with a non-proprietary x86 Windows executable.
3. Reach WoW login screen with bots off, safe renderer, audio off and 960×540/30 FPS.
4. Join local realm and run 30-minute zero-bot session while collecting complete CPU/GPU/memory/thermal data.
5. Tune renderer and resolution before enabling bots. Server load and translation load compete for the same thermal/power envelope.
6. Enable 25 bots and run 2-hour soak; adjust device qualification, not hidden global defaults.
7. Test suspend/resume, Bluetooth controller, IME, phone call/audio focus, low battery and thermal throttling.
8. Repeat on a 64-bit-only device using the WoW64 branch and on a Mali device before claiming general ARM64 support.

21.7 ARM-specific risk register

ID	Risk	Detection	Mitigation
ARM-01	32-bit path unavailable	ABI/runtime self-test	WoW64 branch or mark unsupported.
ARM-02	Box/Wine crash in WoW startup	Native backtrace + translator log	Pinned known-good stack; self-test; branch-specific safe mode.
ARM-03	JIT/cache consumes excessive address space	PSS/VSS/cache metrics	Cache cap, profile tuning, qualified RAM floor.
ARM-04	GPU driver black screen	Renderer self-test/game safe mode	Backend fallback, driver/device profile denylist.
ARM-05	Thermal throttling collapses client/server	Thermal + frame/tick correlation	30 FPS, lower resolution/bots, admission controller.
ARM-06	Mali path much slower than Adreno	Per-device performance gates	Separate recommended profiles; do not generalize one benchmark.
ARM-07	16 KB page-size translator assumption	16 KB device/emulator	Rebuild and remove 4 KB assumptions across all prebuilts.
ARM-08	Vendor kills foreground processes	Long soak/background transitions	User-visible battery-optimization guidance only when necessary; robust recovery.

21.8 Definition of ARM parity

ARM parity is not “the APK installs.” It means the same imported client and database backup can move from the x86 development build to an ARM64 build; the app detects/rebuilds only ABI-specific runtime state; the user can log into the same character; all MVP functional, lifecycle, recovery and touch-input gates pass; and performance is within the published device profile. The database and managed client must never require an architecture-specific destructive migration.

22. Performance, memory, storage and thermal budgets

22.1 Budget philosophy

This workload combines a database, two server processes, bot AI, a Windows compatibility layer, optional CPU translation, graphics translation, a 3D client and an Android UI. Resource limits must be designed as a whole-system envelope. A component benchmark that ignores the rest of the stack is not a release result. Values in this section are planning ranges until replaced by measurements from named device profiles.

22.2 Memory planning envelope

Component	Planning working range	Primary drivers
Android UI + supervisor	100–250 MiB	UI toolkit, logs, import metadata, process bindings.
MariaDB	250–900 MiB	Buffer pool, connections, schema/cache, bot/AH activity.
realmd	30–120 MiB	Small; libraries/log buffers.
mangosd, 0–25 bots	500–1,200 MiB	World data, scripts, maps metadata, bot AI.
mangosd, 50–150 bots	900–2,500 MiB	Bot count/activity, caches, allocator behavior.
Wine + WoW client	1,200–2,500 MiB	Resolution, assets, Wine architecture and renderer.
Box translation/cache on ARM	300–2,000 MiB	Dynamic translation cache/address-space strategy.
GPU/shared graphics memory	300–1,000 MiB	Resolution, textures, DXVK/WineD3D and driver.
Whole-stack practical range	3–7 GiB	Not additive maxima; measure PSS and kill margin.

> **Initial device guidance**
>
> Treat 8 GB physical RAM as a practical minimum for the first supported ARM profile and 12 GB as a more comfortable target, subject to measured PSS, vendor memory policy and bot tier. A 4–6 GB device may run a reduced profile but should not be promised until it passes the same soak and force-stop gates.

22.3 Memory admission rules

- Use PSS/RSS from Android/process diagnostics, not Java memoryClass alone.
- Reserve headroom for the OS, launcher, graphics driver and transient zone loads. Do not target 95% of physical RAM.
- Before starting, compare the selected profile's observed p95 peak against current available memory and a safety margin.
- During play, bot admission may reduce online count, but never shrink critical MariaDB durability buffers dynamically without a controlled restart.
- If the client process is killed by memory pressure, record system evidence and offer a lower profile rather than immediately relaunching into the same condition.
- Keep memory profiles keyed by device/runtime build, because a Wine or renderer update can change consumption materially.

22.4 Storage planning

Area	Typical planning range	Quota/cleanup policy
Managed client	Measured source; often several GiB	Required; user can re-import/delete explicitly.
Server data	1–4 GiB	Versioned; keep active + at most one staged/previous set.
Database	0.5–2+ GiB	Warn on growth; never delete automatically.
Database backups	0.2–2+ GiB each	Retention count/size selected by user; protect latest compatible backup.
Wine prefix/cache	0.5–1.5 GiB	Prefix retained; caches safely clearable.
Translator/shader caches	0.2–2 GiB	Per-build quota; clear on incompatible update.

Area	Typical planning range	Quota/cleanup policy
Logs/support staging	≤ 0.1–0.5 GiB	Rotate; delete exports after share/cancel.
Update/import staging	Up to full changed payload	Preflight exact need; clean abandoned operations.
Practical free storage	8–16 GiB planning target	Calculate from actual source and selected options.

22.5 Storage floor behavior

Threshold	Action
Preflight below required operation bytes	Block operation and show category-by-category requirement.
Runtime free < 2 GiB planning warning	Warn, stop bot growth/AH activity and suggest cache cleanup; tune after measurements.
Runtime free < 1 GiB critical planning floor	Request save and controlled stop unless measured database safety policy allows continuing.
Database reports disk full	Stop world immediately through supported shutdown if responsive; preserve logs; do not retry writes blindly.
Extraction fills disk	Pause/cancel, keep completed checkpoints, remove only current partial output.

22.6 CPU and world-update budgets

Metric	Planning target	Interpretation
World update p50	< 50 ms	Ordinary update loop should have broad headroom.
World update p95	< 150 ms	Transient AI/database work acceptable within observed playability.
World update p99	< 250 ms	Sustained breach triggers bot admission reduction/warning.
Hard stall	> 1,000 ms repeated	Classify unhealthy; capture stacks/DB waits before restart.
Client main-frame p95	≤ selected frame budget	33.3/22.2/16.7 ms by 30/45/60 FPS profile.
Database query latency	Record p50/p95 by query class	No fixed universal target until instrumentation identifies critical paths.

22.7 Thermal and power policy

- Capture Android thermal status, battery temperature where permitted, charging state, CPU frequency snapshots and frame/world performance correlation.
- At MODERATE thermal status, stop increasing bots and reduce optional extraction/background work.
- At SEVERE, pause extraction, lower dynamic bot target and offer/trigger a lower frame cap according to user setting.
- At CRITICAL/EMERGENCY, save and stop if the platform has not already intervened. Do not attempt to outsmart thermal protection.
- Do not hold a wake lock for an idle stopped realm. During play, keep only the minimum locks required by actual behavior.
- Test plugged-in and battery operation. Charging heat can make a nominally faster profile less stable.

22.8 Reference scenes for repeatable profiling

Scene	Purpose	Metrics
PERF-S01 Character select	Renderer/client baseline	FPS, PSS, GPU, audio initialization.
PERF-S02 Quiet starter town	Ordinary world/client baseline	Frame time, world tick, total CPU, 0/25 bots.
PERF-S03 Dense capital	NPC/object/render pressure	Frame time, memory, shader cache behavior.
PERF-S04 Combat with 4 party bots	AI + effects + input	Tick p99, frame p95, DB/loot latency.
PERF-S05 100 random bots ramp	Server/bot stress	RSS/PSS, tick, login rate, SQL activity.
PERF-S06 2-hour questing route	Thermal/long soak	Thermal, battery, FPS/tick drift, leaks.

Scene	Purpose	Metrics
PERF-S07 Restart loop	Cache/startup stability	20 start/stop durations and peak memory.

22.9 Performance data record

Minimum local CSV/JSON performance schema

```
profileId, deviceIdHash, androidBuild, appBuild, runtimeManifestId,
clientBackend, renderer, virtualResolution, fpsCap, botProfile,
sessionId, sceneId, startElapsed, durationSec,
appPssMiB, dbPssMiB, realmPssMiB, worldPssMiB, clientPssMiB,
frameP50Ms, frameP95Ms, frameP99Ms,
worldP50Ms, worldP95Ms, worldP99Ms,
thermalMax, batteryDeltaPct, freeStorageMinMiB, outcome
```

23. Security and privacy requirements

23.1 Threat model

Threat	Control
Malicious or malformed imported tree	SAF-scoped read, path normalization, type/size limits, PE/data validation, no execution of imported helpers.
Tampered runtime/update	APK signing, signed compatibility manifest, SHA-256 verification, immutable executable code.
Local app attacks on exported components	Non-exported services/providers, signature permissions where needed, unguessable session tokens.
Database/control exposure	Unix sockets/loopback, restrictive permissions, no default TCP admin or RA service.
Credential leakage	No command-line/env passwords, structured redaction, Keystore-backed optional storage, zeroization best effort.
Unsafe native code	Process isolation, fuzzed parsers/control protocol, bounded inputs, modern compiler hardening and native symbols.
Supply-chain drift	Pinned commits, SBOM, patch review, reproducible builders and signed artifacts.
Support bundle privacy	Explicit preview/consent, redaction tests, no full databases/client files by default.
LAN exposure	Off by default; separate permissions/security review; no automatic port forwarding.

23.2 Imported-content rules

- The only imported executable launched is the validated WoW.exe at the expected relative location. Do not execute source launchers, patchers, installers, batch files or DLL injectors.
- Default safe mode ignores unknown DLL overrides and appinit-style additions. Display detected customization warnings.
- Reject files with traversal paths, impossible sizes, unsupported special types or case-insensitive collisions.
- Bound archive/decompression ratios if any archive import is later supported; folder import is simpler and preferred.
- Treat MPQ and PE parsing as untrusted-input code. Run high-risk extraction/parser work in :worker with memory/time limits and fuzz tests.
- Do not upload hashes, paths or content unless the user explicitly exports a support bundle.

23.3 Secret handling

Secret	Storage/transport
Local WoW account password	Not stored by default. Optional encrypted credential via Android Keystore; passed to world control out-of-band/in memory.
Database root/admin secret	Generated randomly, encrypted app-private storage, never exposed to UI.
Core DB user secret	Generated separately, supplied via protected config/descriptor; redacted everywhere.
Update signing keys	Never in app/repo; offline or protected CI signing service.
Support bundle encryption key	User-provided/passphrase-derived or platform sharing flow if encrypted export is offered.

23.4 Native hardening

- Compile with stack protector, fortified source where compatible, RELRO, non-executable stack, PIE/shared-library conventions and hidden default symbol visibility.
- Enable sanitizers in emulator/debug variants for import parsers, control protocol and adapted server code; do not ship sanitizer runtimes by accident.
- Run static analysis and dependency vulnerability scanning, but adjudicate relevance to the pinned Android build.
- Retain separate unstripped symbols keyed by GNU build ID; strip release binaries.
- Use seccomp only if carefully compatible with Wine/server behavior; Android already applies app sandbox/SELinux. Do not add brittle filters before syscall profiling.

- Fuzz the local control JSON framing, path manifest parser, PE/version parser and any MPQ/extractor wrapper boundary.

23.5 Network minimization

A local game session should work in airplane mode after installation and setup. Add a test that blocks external network access and asserts that only loopback sockets are used. Optional update checks must be user-visible, TLS-protected and separate from game startup. Never send telemetry by default; local performance data can be exported manually.

23.6 Security release checklist

Gate	Required result
SEC-G01	No Blizzard/proprietary client file appears in APK, test fixture or update channel.
SEC-G02	APK and update manifests verify; tampered runtime artifact is rejected before load.
SEC-G03	All services/providers/receivers reviewed for exported state and intent validation.
SEC-G04	Loopback/socket bind scan shows no unexpected listening interface.
SEC-G05	Synthetic passwords/DB secrets absent from logs and support bundle.
SEC-G06	Path traversal/case collision/special file and oversized import tests pass.
SEC-G07	Release native binaries pass hardening and 16 KB alignment checks.
SEC-G08	SBOM, notices, source obligations and patch provenance complete.
SEC-G09	Debug menus, verbose admin sockets and test keys absent from release.
SEC-G10	Backup/export restore and optional encryption tested.

24. Reproducible build, updates and release engineering

24.1 Reproducible builder

Build all open-source native components in a pinned container/VM image, not on the Android device and not from whatever packages happen to be installed on the Windows development host. Store builder image digest, tool versions and environment normalization. CI should rebuild at least one release from source and compare artifact hashes or provide a deterministic-difference report.

Pinned input	Examples
Host builder image	Linux distribution/image digest; packages locked.
Android toolchain	JDK, Gradle wrapper, Android Gradle Plugin, SDK build tools, NDK, CMake, Ninja.
Source	Full commits and submodule hashes for core, bots, DB, Wine, Box, Mesa/DXVK and dependencies.
Patches	Ordered patch hash set and generated-source scripts.
Compiler flags	Per ABI/build type; page-size/linker settings.
Generated manifests	SBOM, notices, runtime compatibility and database migration graph.
Signing	Unsigned artifact reproducibility separated from protected signing step.

24.2 Release artifact strategy

Artifact	Content	Recommendation
arm64-v8a APK	App + ARM native server + ARM client runtime/translator	Primary physical release.
x86_64 APK	App + x86_64 server + x86 client runtime as selected	CI/emulator and niche device use.
Universal APK	Both major ABI payloads	Optional; publish only if size/install behavior is acceptable.
AAB	ABI split delivered by store	Potential Play route after policy/legal review.
Symbols archive	Unstripped native symbols and mapping files	Private release artifact keyed by build ID.
Source/patch bundle	Corresponding open-source code and build scripts	Publish/offer as license requires.
SBOM/notices	Machine and human-readable dependency record	Ship with every release.

24.3 Update atomicity

Runtime executable code updates through APK installation. Mutable data/schema updates are staged after the new APK starts and before the realm runs. The app must support a failure between APK update, database migration, data extraction and first successful world start without destroying the last compatible state.

1. New APK verifies current managed-data manifests and determines required migration plan.
2. Create/verify pre-update database backup and ensure free staging space.
3. Stage data/config/prefix changes under new version IDs. Do not switch active pointers yet.
4. Stop all runtime components and apply database migrations transactionally where supported.
5. Run offline compatibility checks and a headless/server startup smoke if feasible.
6. Switch active manifests/pointers and mark UPDATE_PENDING_FIRST_RUN.
7. Start realm/client. After reaching the defined successful milestone, mark update committed and allow old staging cleanup.
8. On failure, stop, preserve logs and restore the compatible database/data/prefix set where reversible. APK downgrade may require user reinstall and is not assumed automatically.

24.4 Compatibility classes

Change	Class	Required action
UI-only fix	C0	No runtime migration; smoke start.
Server/runtime binary, same schema/data	C1	Backup recommended; full startup/soak regression.

Change	Class	Required action
Config schema change	C2	Typed migration + rollback of rendered config.
Wine/renderer update	C3	New prefix/cache staging or compatibility test; retain old prefix until success.
Core/database SQL migration	C4	Mandatory backup, pinned migration ledger, restore test.
Client-derived data format change	C5	Stage/rebuild data set; retain prior set while compatible APK remains.
Client build target change	C6	New product manifest/import; not an in-place Vanilla v1 migration.

24.5 Release channels

Channel	Audience	Rules
dev	Agent/developers	Debug logs, fault injection, unsigned/dev key; data disposable.
canary	Internal device lab	Signed test key, migration from prior canary, telemetry local/exported.
beta	Informed testers	No proprietary assets; documented supported devices; backup required before upgrade.
stable	Qualified users	All release gates, legal/license review, reproducible artifact and rollback/restore path.

24.6 Release notes must state exact support

- Supported client: World of Warcraft 1.12.1 build 5875, user-supplied.
- Supported Android APIs, ABIs, page sizes and tested device/GPU profiles.
- Client backend, Wine/Box/renderer build IDs and known renderer fallbacks.
- Default and maximum qualified bot profiles by device class.
- Database/core/playerbot commits and migration class.
- Known limitations: touch controls, extraction duration/storage, LAN off, unsupported custom launchers/addons.
- Backup requirement and exact update/restore behavior.

24.7 CI matrix

Job	Frequency	Key result
Source/license/SBOM validation	Every change	No unpinned or unclassified dependency.
Cross-build x86_64/arm64	Every change affecting native/runtime	No host contamination; symbols archived.
Unit/static/fuzz smoke	Every change/nightly	Control/import parsers and core adaptations.
AVD legacy smoke	Every client/runtime change	Wine/client login screen and process lifecycle.
AVD modern integration	Nightly/release	DB→world→client loop, no bots, clean persistence.
16 KB AVD	Nightly/release	All native code loads and integration smoke.
Bot soak	Nightly/weekly depending cost	25/50/100 profiles with metrics.
Forced-stop/recovery	Nightly/release	Deterministic fault points restore safely.
Physical device matrix	Release candidate	Adreno, Mali, 64-bit-only, 8/12 GB profiles.
Reproducible rebuild	Release candidate	Match or documented deterministic differences.

25. Verification plan and acceptance gates

25.1 Test layers

Layer	Examples	Environment
Unit	Path normalization, manifest parsing, state transitions, redaction, profile bounds	Host JVM/native and Android instrumentation.
Component	MariaDB start/recovery, realmd/world control, Wine prefix self-test	AVD per ABI/API.
Contract	Binder/control JSON/versioning/error mappings	Fake peer + real component.
Integration	Imported client → local login → character persistence	x86 AVD first, then ARM devices.
Fault injection	Kill at every startup/stop/import/migration boundary	Debug AVD/device.
Performance	Reference scenes, bot tiers, frame/world timing	Fixed AVD and named physical profiles.
Usability	Touch-only task completion and accessibility	Phone/tablet aspect ratios.
Security	Malformed import, tampered update, secret export, bind scan	AVD/device and CI fuzzers.
Upgrade/restore	Prior stable → candidate, backup restore, prefix/data staging	Clean and populated fixtures.

25.2 Release-blocking functional test matrix

ID	Scenario	Targets	Pass
FUN-001	Fresh install → import build 5875 → prepare → account → first login	x86 modern + ARM reference	Pass end to end without shell/manual file editing.
FUN-002	Import wrong client build	All	Block before copy or after fast scan with exact detected build.
FUN-003	Interrupt import at 10%, 50%, final rename	All	Resume 10/10; final manifest correct.
FUN-004	Interrupt each extraction stage	All	Resume/checkpoint; no incomplete active data.
FUN-005	Create normal and GM account	All	Core command path; correct level; no secret logs.
FUN-006	Zero-bot 30-minute play and restart	All	Character state persists.
FUN-007	25-bot 2-hour play/idle	Reference profiles	No fatal/OOM; metrics within limits.
FUN-008	Touch UX-T01–T08	Phone aspect	All tasks complete.
FUN-009	Physical keyboard/mouse/gamepad	Supported peripherals	No stuck keys; mapping persistence.
FUN-010	Relaunch client while realm remains	All	Returns to account/character without realm corruption.
FUN-011	Create and restore backup	All	Exact selected character assertions restored.
FUN-012	Airplane/offline mode	All	Full local play; no external dependency.

25.3 Startup/shutdown fault matrix

ID	Injected failure	Required result
FLT-01	Kill app during DB initialization	Idempotent resume or reset; no partially “ready” manifest.
FLT-02	Kill MariaDB during normal play	Realm stops/fails; next launch recovers or offers restore.

ID	Injected failure	Required result
FLT-03	Kill realmd during world session	Unhealthy state recorded; safe whole-stack restart path.
FLT-04	Kill mangosd during save	Dirty journal; DB recovery; no silent clean mark.
FLT-05	Kill client only	Realm retained for bounded relaunch; state not lost.
FLT-06	Kill supervisor	No uncontrolled orphan; component/session ownership recovered.
FLT-07	Force-stop Android app	No resurrection; next explicit launch recovers.
FLT-08	Port 3724/8085 occupied	Do not kill unknown process; actionable PORT_IN_USE.
FLT-09	Storage full during DB write	Controlled stop/evidence; no retry loop.
FLT-10	Storage full during import/extraction	Partial only; resume after space.
FLT-11	Source URI permission revoked mid-import	Pause with permission error; managed verified files retained.
FLT-12	Renderer crashes twice	Offer deterministic safe mode; no infinite relaunch.
FLT-13	Audio backend blocks/fails	Audio-off retry; realm remains healthy.
FLT-14	Migration SQL fails halfway	Transaction/ledger shows failure; restore pre-snapshot.

25.4 Quantitative soak suite

Suite	Duration/count	Configuration	Acceptance
SOAK-START	20 cold/warm cycles	0 bots, safe renderer	20 clean starts/stops; no data loss; duration trend recorded.
SOAK-IMPORT	10 interruptions	Representative client tree	Resume and final manifest match.
SOAK-ZERO	2 hours	0 bots	No leak trend/fatal; playability limits pass.
SOAK-25	2 hours	25 bots	Tick/memory/thermal limits pass.
SOAK-50	2 hours	50 bots	Qualified profile only; compare to 25.
SOAK-100	2 hours	100 bots	Advanced profile; no universal release block if not advertised.
SOAK-LONG	6 hours	Advertised default profile	No corruption, runaway logs/cache or unbounded memory growth.
SOAK-RECOVER	10 forced stops	Different lifecycle points	10 safe recoveries or explicit restore path.
SOAK-THERMAL	120 minutes battery + charging	Default ARM profile	No critical thermal; stable adaptation.

25.5 Database/state assertions

- Selected character exists with expected name, level, position/map, inventory sentinel item and quest state after each restart/restore test.
- Account count and bot-generated character ranges do not duplicate after interrupted generation.
- Core/database revision tables match compatibility manifest.
- Realm row has exactly one expected local realm and correct address/port/build settings.
- MariaDB reports clean shutdown when expected and no corruption warnings after recovery suite.
- Backup restore into a fresh datadir reaches world-ready before replacing the active datadir.

25.6 Compatibility matrix to publish

Dimension	Minimum coverage before stable
Android API	Research API 28; selected minimum; current target; Android 15+ 16 KB.
ABI	x86_64 development artifact; arm64-v8a release; x86 only if retained.

Dimension	Minimum coverage before stable
ARM userspace	At least one 64-bit-only device and one device/path representing any 32-bit branch claimed.
GPU	At least two Adreno profiles and one Mali profile; unsupported families stated.
RAM	8 GB and 12 GB reference; lower only if advertised.
Storage provider	Internal Documents, common file manager provider, removable storage if claimed.
Input	Touch-only, Bluetooth gamepad, keyboard/mouse.
Audio	Speaker, wired/USB where available, Bluetooth.
Orientation/aspect	At least 16:9, 20:9 and tablet if tablet support is claimed.

25.7 Exit gates by phase

Gate	Required before proceeding
G0 Architecture	ADR-001–012 accepted; production packaging spike has a viable path.
G1 Client x86	WoW build 5875 login UI under direct Wine; input and safe renderer.
G2 Server x86	Native DB/realmd/world no-bot; control and clean persistence.
G3 Integrated x86	FUN-001/FUN-006, 20 cycles, forced recovery and importer.
G4 Bots/UX	25-bot soak and UX-T01–T08.
G5 ARM	Native server parity, translated client, 0/25-bot and physical-device matrix.
G6 Release	Modern target/16 KB, security/license, update/restore and reproducible build.

26. Indexed issue and troubleshooting catalogue

Use the stable IDs below in agent notes, commits, UI error mappings and support bundles. “First action” is deliberately narrow; preserve evidence before applying broad resets. Severity: S0 data/security critical, S1 release-blocking, S2 major, S3 minor/diagnostic.

ID	Sev	Symptom	Likely cause	First action	Ref.
PLAT-001	S1	Native payload works on target 28 but fails on modern target	Executable unpacked into writable app home	Prove APK-native/library packaging; inspect denial	5.1, 8.4
PLAT-002	S1	Foreground service throws at start	Missing type/permission or illegal background start	Start from visible user action; verify manifest/runtime type	5.3, 8.3
PLAT-003	S2	Runtime stops after hours on Android 15	Misclassified dataSync/mediaProcessing service	Use appropriate visible runtime model; inspect timeout callback	5.3
PLAT-004	S1	Native library fails only on 16 KB image	ELF alignment or 4 KB code assumption	Check APK alignment, DT_LOAD and mmap/page constants	5.5
PLAT-005	S2	Service survives UI but dies after task/user action	Lifecycle/process-importance policy	Inspect system logs; verify FGS and session journal	18
ABI-001	S1	INSTALL_FAILED_NO_MATCHING_ABIS	APK lacks AVD/device ABI	Capture abilist; build correct split	2.2, 20.1
ABI-002	S1	WoW.exe reports bad format	Wine lacks 32-bit/WoW64 path	PE I386 check and Wine capability self-test	15.4
ABI-003	S1	Stock Winlator module cannot load on x86 AVD	ARM64-only native payload/assumptions	Use direct x86 Wine backend	15.2
ABI-004	S1	ARM device has no 32-bit support	64-bit-only platform	Select Box64+Wine WoW64 or mark unsupported	21.2
ABI-005	S2	Works on one ARM device, crashes on another	Translator/driver/vendor variance	Compare capability/runtime build; qualify device profile	21
STO-001	S2	Folder picker cannot select Download/root	Android 11 SAF restriction	Guide user to Documents/Games folder	5.2, 9.1
STO-002	S1	Import loses access after restart	URI permission not persisted/provider revoked	Persist grant; prompt reselect; retain verified staging	9
STO-003	S0	Source files modified	Runtime pointed at source or importer wrote config	Enforce managed copy; audit write roots	9.7
STO-004	S1	Import resumes with corrupt partial file	Journal/atomic rename missing	Delete/revalidate partial; use temp+fsync+rename	9.5–9.6
STO-005	S1	Two source files overwrite each other	Case-fold collision/normalization	Reject collision during discovery	9.2
STO-006	S1	Setup fails late with disk full	Incomplete storage preflight/staging margin	Recalculate categories and allocatable bytes	9.4, 22.4
STO-007	S2	Game I/O extremely slow from imported folder	Running through document provider	Copy to app-private managed client	5.2, 9
CLI-001	S1	Client rejected as unsupported	Not build 5875 or identity parser weak	Inspect PE/version/data; no normal bypass	9.2–9.3
CLI-002	S1	Login screen never appears	Wine loader/prefix/renderer failure	Run Wine self-test then client safe mode	15
CLI-003	S1	Black window	DXVK/WineD3D/surface issue	Switch to WineD3D, 720p, audio/overlay off	15.8, 16.1
CLI-004	S2	Client works only with source launcher	Custom launcher/injection dependency	Reject normal import; try clean direct WoW.exe source	15.10, 23.2
CLI-005	S2	Crash after addon loading	Incompatible/custom addon	Temporarily disable addons; preserve files	16.9
CLI-006	S2	Repeated realm-list screen	Realm row/address/build/WDB mismatch	Verify loopback/ports/build; clear cache only after evidence	17.6
CLI-007	S2	Credentials accepted then disconnected	World not ready or protocol/build mismatch	Correlate realmd/world logs and readiness	15.8, 17
DATA-001	S1	World reports missing maps/DBC	Incomplete or wrong data directory	Validate manifest; re-extract required stages	10
DATA-002	S2	Bots/NPCs fail line of sight/pathing	Missing/corrupt vmaps/mmaps	Coverage report and targeted re-extraction	10.4–10.5
DATA-003	S2	MMap generation appears frozen	Long tile or thermal throttle	Expose tile progress; inspect CPU/thermal; checkpoint	10.3
DATA-004	S1	Extractor output published incomplete	No atomic data manifest switch	Revoke active set; stage/validate/publish	10.3–10.4
DATA-005	S1	Data works with old core, fails after update	Extractor/core format drift	Compare manifest IDs; rebuild staged set	24.4

ID	Sev	Symptom	Likely cause	First action	Ref.
DB-001	S1	mariadbd cannot load libmariadb/dependency	Incorrect APK/NDK linkage	Inspect DT_NEEDED and ABI; fix CMake target	12.7
DB-002	S1	Core cannot connect to running DB	Socket path/permission/config mismatch	Connect with same user/socket from service probe	12.2–12.3
DB-003	S1	Required database update/revision error	Mixed commits or unapplied SQL	Stop and compare lock manifest/ledger	12.5
DB-004	S0	Database recovery loops after force-stop	Corruption/unsupported recovery/repeated restart	Preserve datadir/log; stop retries; restore/advanced recovery	12.7, 19.6
DB-005	S1	Database killed under bot load	Memory pressure/buffer profile	Reduce bots/buffer; capture PSS/system kill reason	12.7, 22
DB-006	S0	Migration failed after partial changes	Nontransactional SQL or missing snapshot	Restore pre-snapshot; fix idempotency/ledger	12.5, 24.3
SRV-001	S1	realmd process exists but port closed	Startup not ready/config failure	Use structured ready and inspect realmd log	11.7, 17.3
SRV-002	S1	world process exists but client cannot enter	World not fully ready/data/script failure	Require world-ready event plus 8085 probe	11.7
SRV-003	S1	Address already in use	Stale known session or unrelated process	Verify instance token; never kill unknown listener	18.6
SRV-004	S2	World hangs during startup	DB wait/data load/bot generation	Last structured milestone, thread stacks, query/tick trace	18.2
SRV-005	S0	Character state lost after normal stop	Shutdown/save order or DB durability fault	Audit save acknowledgment, DB checkpoint and clean journal	18.3
BOT-001	S2	First bot generation takes very long	Large unbounded batch/SQL work	Batch, checkpoint, progress; start at 25	13.4
BOT-002	S1	World tick collapses as bots log in	Login storm/too-high target	Ramp small batches; admission controller	13.4–13.5
BOT-003	S2	Bots attack neutralized/invalid targets repeatedly	AI behavior defect at pinned revision	Reproduce scripted scenario; upstream/local patch with test	13.7
BOT-004	S2	Looting slows server	Bot loot strategy/DB pressure	Disable/simplify profile; profile queries/tick	13.7
BOT-005	S2	Bots stuck in water/terrain	MMap/vmap coverage or AI pathing	Validate data then isolate core AI issue	10.5, 13.6
BOT-006	S1	Bot downgrade does not reduce load	Persistent target/logouts not applied	Gradual supported logout command; observe online count	13.4
BOT-007	S1	AHBot fills storage/DB or stalls updates	Enabled too early/high volume	Disable; restore/cleanup supported data; separate qualification	13
WINE-001	S1	wineboot/prefix init fails	Code packaging, architecture or writable-prefix issue	Run minimal self-test; inspect loader/prefix permissions	15.5
WINE-002	S2	Old prefix breaks after runtime update	Incompatible Wine registry/components	Stage new prefix; retain old until success	15.5, 24.4
WINE-003	S1	wineserver remains after client close	Shutdown/session ownership fault	Token-scoped close then bounded force stop	18.6
WINE-004	S2	Verbose logs fill storage	WINEDEBUG channels left enabled	Bound session log and reset default channels	19.2
GPU-001	S1	DXVK black screen/crash	Vulkan/driver incompatibility	Renderer self-test; fallback WineD3D	16.1
GPU-002	S2	Very low FPS on emulator	Host GPU mode/WineD3D translation/resolution	Record AVD GPU mode; lower res; compare Vulkan	20.7
GPU-003	S2	Adreno works, Mali fails	Turnip-specific path or system driver issue	Select Mali profile/fallback; do not apply Turnip	21.5
GPU-004	S2	Stutter after update	Shader/translation cache invalidation	Clear incompatible cache; compare build IDs	24.4
AUD-001	S2	Crackle/high latency	Buffer/route/backend	Record underruns; select larger stable buffer	16.5
AUD-002	S2	Client hangs at audio init	Backend deadlock/failure	One audio-off safe retry	16.5
INP-001	S1	Character keeps moving after touch/IME	Missing synthetic key-up	Release all on focus/profile/IME transitions	16.8
INP-002	S2	Touch clicks wrong UI position	Scale/letterbox transform error	Log/display coordinate calibration and matrix test	16.3, 16.6
INP-003	S2	Chat impossible or duplicates text	IME bridge semantics	Explicit chat field; test commit/Enter once	16.6–16.10
INP-004	S2	Overlay covers critical UI	Layout/scale not responsive	Compact/custom profile; task-based UX test	16.7
INP-005	S2	Controller mapping changes by device	Axis/button/vendor variance	Per-device calibration and profile IDs	16.6

ID	Sev	Symptom	Likely cause	First action	Ref.
NET-001	S1	Using 127.0.0.1 reaches wrong machine in test	Host/guest address confusion	Guest loopback is guest; host is 10.0.2.2	17.4
NET-002	S1	Server listens on all interfaces	Default BindIP copied	Force loopback/socket and verify bind scan	17.1, 23.5
NET-003	S2	VPN/private DNS interferes	Network namespace/routing side effect	Airplane/loopback test; capture routes/sockets	17
LIFE-001	S0	App marks stop clean after forced kill	Journal written optimistically	Set clean=false at session start; clean only last	18.5
LIFE-002	S1	Stop hangs indefinitely	No bounded component timeout	Apply staged shutdown and escalation	18.2–18.3
LIFE-003	S1	Old PID causes wrong process kill	PID reuse/no token	Require session/instance token handshake	18.6
LIFE-004	S2	Realm silently restarts after user stop	Scheduler/receiver resurrection	Remove auto-resurrection; require explicit Start	18.7
BLD-001	S1	Android build links host library	CMake search contamination	Hermetic sysroot/imported targets; inspect binary	7
BLD-002	S1	Community flag hides duplicate symbols	--allow-multiple-definition workaround	Remove flag; fix symbol ownership/link order	7.4
BLD-003	S1	Rebuild changes component unexpectedly	Unpinned branch/latest download	Full commit/hash lock and offline build	7.3
BLD-004	S1	Release lacks source/license obligations	Incomplete provenance/SBOM	Block release until inventory complete	4.2–4.3
UPD-001	S0	Update migrates DB but old app cannot run	Irreversible C4 migration	Pre-snapshot; explicit rollback/restore plan	24.3–24.4
UPD-002	S1	New Wine corrupts/reuses old prefix	No prefix compatibility class	Stage versioned prefix and first-run commit	24.4
UPD-003	S1	Update deletes active data during failure	In-place overwrite	Versioned staging and atomic pointer switch	24.3
SEC-001	S0	Imported malicious helper is executed	Launcher/installer trusted from source	Only execute validated WoW.exe; ignore helpers	23.2
SEC-002	S0	Password appears in log/export	Command-line/env/raw logging	Protected channel, structured redaction and canary test	23.3
SEC-003	S0	Admin/database reachable from LAN	Bind/export misconfiguration	Unix socket/loopback; port scan release gate	23.5
SEC-004	S1	Tampered runtime loads	Unsigned/unverified mutable code	APK-native code + signed manifest verification	23.1

26.1 Issue handling protocol

1. Record issue ID, app/runtime manifest IDs, device/AVD profile and session ID.
2. Preserve the failed session logs/journal and current manifests before reset or re-import.
3. Reduce to the earliest failing layer: packaging → runtime self-test → client login UI → DB → realm → world → integrated login → bots → renderer/input enhancements.
4. Apply the narrow first action. Do not combine cache clear, prefix reset, DB restore and client re-import in one attempt.
5. After resolution, add a regression test and link the patch/ADR to the stable issue ID.
6. When evidence comes only from a community report, label the test hypothesis and record whether it reproduced on the pinned stack.

27. Dependency-ordered delivery roadmap

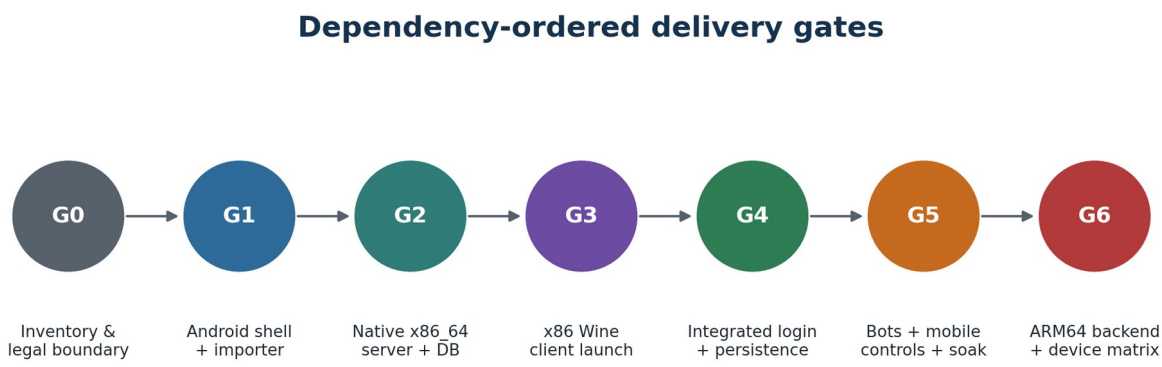

Figure 3. Delivery proceeds by dependency gates rather than calendar promises; each gate leaves a testable, recoverable system.

27.1 Workstreams and deliverables

Gate	Deliverables	Exit evidence
G0 Architecture proof	ADR set, legal boundary, compatibility lock, production packaging experiments PKG-01–06	A modern-target native/Wine packaging route is technically viable.
G1 x86 client	Direct Wine backend, prefix manager, surface/input bridge, build-5875 importer fast scan	WoW login screen on fixed x86 AVD; safe mode works.
G2 x86 native realm	MariaDB, realmd, mangosd no bots, control socket, manifests	World-ready/save/stop/recovery without client.
G3 integrated app	Supervisor, one-action start/stop, full importer/extractor, account UI, diagnostics	Local play/persistence and 20-cycle gate.
G4 bots and mobile UX	25/50 profiles, admission controller, touch/gamepad/IME, backups	25-bot soak and UX acceptance.
G5 ARM64	Native server ABI, Box/Wine branches, Adreno/Mali render profiles	ARM parity on qualified devices including 64-bit-only.
G6 release engineering	Modern target, 16 KB, signed updates, SBOM/notices, security and migrations	Stable-candidate gate.

27.2 Parallelizable versus sequential work

Can proceed in parallel	Must remain sequential
UI shell and state visualization using fake RuntimeController	Do not integrate client and server before each passes independently.
Importer path safety/journal and client PE/build parser	Do not run extraction before import validation and storage preflight.
MariaDB Android port and direct x86 Wine proof	Do not enable playerbots before no-bot persistence/recovery.
Input profile editor and logical event mapping	Do not tune ARM graphics before translator/Wine self-test.
SBOM/license inventory from first commit	Do not make public binaries before legal/license gate.
Test harness/fault injection and structured log schema	Do not implement updater before data/schema compatibility model.

27.3 Agent task breakdown

Task	Deliverable	Gate	Done when
A-001	Create component lock manifest and repository/submodule skeleton	G0	Manifest validates full commits and licenses.

Task	Deliverable	Gate	Done when
A-002	Build capability-report Android app and AVD definitions	G0	Support bundle matches adb.
A-003	Complete PKG-01/02 native execution experiments on API 28/current/16 KB	G0	Documented chosen production lane.
A-004	Build direct x86 Wine self-test and surface bridge	G1	Test window renders and accepts input.
A-005	Implement SAF fast scan and PE/build 5875 validator	G1	Wrong builds rejected; source untouched.
A-006	Launch imported WoW.exe to login UI in safe mode	G1	Repeatable X3 artifact/logs.
A-007	Cross-build MariaDB x86_64 with app-private socket	G2	Init/query/clean/dirty recovery.
A-008	Port realmd then mangosd no-bot with control/log modules	G2	Structured readiness and clean stop.
A-009	Pin/import classic DB and implement migration ledger	G2	Revision mismatch and upgrade tests.
A-010	Implement RuntimeSupervisor journal/state machine	G3	Fault-injected transition tests.
A-011	Implement resumable managed copy and data extraction pipeline	G3	10 import interruptions and staged data publish.
A-012	Join client/server/account and pass X7/X8	G3	30-minute play + 20 cycles.
A-013	Implement backup/restore and support bundle	G3	Fresh restore reaches world-ready; redaction canary.
A-014	Enable playerbots 25 tier and instrument world tick	G4	2-hour soak.
A-015	Implement touch/gamepad/IME acceptance tasks	G4	UX-T01–08.
A-016	Build native arm64 server stack from same patches	G5	Server component parity.
A-017	Implement ARM-B WoW64 path, then ARM-A if justified	G5	WoW login/play on qualified devices.
A-018	Qualify Adreno/Mali profiles and bot/resource limits	G5	Published device matrix.
A-019	Implement signed APK/data migrations and rollback workflow	G6	C0–C5 update tests.
A-020	Complete 16 KB, security, license, reproducibility and release gates	G6	Release-candidate evidence pack.

27.4 Stop conditions and architecture review triggers

- If no production-compliant Wine/native-code packaging path works on the current target SDK, stop feature work and resolve packaging rather than shipping a target-28 dead end.
- If direct x86 Wine cannot render the unmodified client reliably, do not begin ARM translation; isolate Wine/client/graphics first.
- If native MariaDB/CMaNGOS cannot survive forced-stop recovery, do not add bots or update features.
- If 25 bots make the world/client exceed an 8 GB reference profile, revise the product default and device support statement instead of hiding the measurement.
- If derived-data distribution is contemplated, trigger legal review before building a download service.
- If a requested feature requires patching proprietary client code, create a separate legal/security ADR and do not blend it into routine import.

27.5 Recommended first coding move

Begin with two independent executable proofs

On the fixed x86 AVD, prove (1) an APK-owned direct x86 Wine backend can display and control the user-supplied build-5875 login screen, and (2) an APK-owned native MariaDB service can initialize, accept a query, stop cleanly and recover after a kill. These two proofs retire the highest uncertainty without entangling client and server debugging.

Appendix A. Configuration and manifest templates

A.1 Android manifest skeleton

Illustrative only: reconcile exact declarations with the selected target SDK and distribution policy

```
<manifest ...>
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />
  <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
  <!-- WAKE_LOCK only if bounded runtime measurements justify it. -->

  <application
      android:allowBackup="false"
      android:extractNativeLibs="<chosen packaging model>"
      android:usesCleartextTraffic="false">

    <service
        android:name=".runtime.RuntimeService"
        android:process=":supervisor"
        android:exported="false"
        android:foregroundServiceType="specialUse">
      <property
          android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE"
          android:value="User-started local game runtime supervising an on-device realm and
compatibility client" />
    </service>

    <service android:name=".native.DatabaseService"
             android:process=":database" android:exported="false" />
    <service android:name=".native.RealmService"
             android:process=":realm" android:exported="false" />
    <service android:name=".native.WorldService"
             android:process=":world" android:exported="false" />
    <service android:name=".client.ClientRuntimeService"
             android:process=":client" android:exported="false" />
  </application>
</manifest>
```

A.2 Effective local realm configuration

Normalized product configuration; render upstream config files from this typed model

```
product.realm.id=1
product.realm.name=Android Local Realm
product.realm.mode=loopback

realmd.bind=127.0.0.1
realmd.port=3724
world.bind=127.0.0.1
world.port=8085

mariadb.networking=false
mariadb.socket=<app>/database/run/mariadb.sock

client.build=5875
client.realmlist=127.0.0.1

bots.profile=mobile-low-v1
auctionHouse.enabled=false
runtime.fpsCap=30
runtime.virtualDesktop=1280x720
runtime.renderer=WINED3D
```

A.3 Managed-client manifest

Source URI/path details should be redacted from support exports

```
{
  "schema": 2,
  "clientId": "uuid",
  "source": {
    "provider": "<authority>",
    "sourceFingerprint": "<hash>",
    "importedAt": "2026-08-01T00:00:00Z"
  },
  "identity": {
```

```
    "version": "1.12.1",
    "build": 5875,
    "peMachine": "I386",
    "wowExeSha256": "<hash>",
    "locale": "enUS"
  },
  "files": {
    "count": 12345,
    "bytes": 5678901234,
    "manifestSha256": "<hash>"
  },
  "appOwnedChanges": [
    {"path":"realmlist.wtf","template":"local-loopback-v1"},
    {"path":"WTF/Config.wtf","keys":["gxResolution","gxWindow","Sound_EnableAllSound"]}
  ],
  "status": "ACTIVE"
}
```

A.4 Server-data manifest

Publish ACTIVE only after required validation

```
{
  "schema": 1,
  "id": "data-<hash>",
  "clientBuild": 5875,
  "coreCommit": "<40-hex>",
  "extractorBuildId": "<hash>",
  "outputs": {
    "dbc": {"complete":true,"files":<n>,"bytes":<n>},
    "maps": {"complete":true,"files":<n>,"bytes":<n>},
    "vmaps": {"complete":true,"files":<n>,"bytes":<n>},
    "mmaps": {"complete":true,"tilesExpected":<n>,"tilesPresent":<n>,"gaps":[]}
  },
  "validation": {"sampleDigest":"<hash>","validatedAt":"..."},
  "status": "ACTIVE"
}
```

A.5 Runtime profile

Profiles are signed/product-owned; user settings may choose among qualified profiles

```
{
  "id": "balanced-arm64-adreno-v1",
  "requirements": {"minRamMiB":8192,"minFreeStorageMiB":2048},
  "client": {
    "backend":"armTranslatedWine",
    "renderer":"DXVK_TURNIP",
    "fallbackRenderer":"WINED3D",
    "width":1280,"height":720,"fpsCap":30,
    "audioProfile":"stable"
  },
  "server": {
    "dbBufferPoolMiB":384,
    "botProfile":"mobile-low-v1",
    "worldTickHardLimitMs":250
  },
  "thermal": {"reduceBotsAt":"SEVERE","pauseWorkersAt":"SEVERE"}
}
```

A.6 Error result schema

Stable error codes bridge UI, logs and Section 26

```
{
  "operationId":"uuid",
  "sessionId":"uuid-or-null",
  "stage":"WORLD_STARTING",
  "errorCode":"DB_REVISION",
  "severity":"BLOCKING",
  "summary":"World database revision does not match this server build.",
  "details":{"expected":"...","actual":"..."},
  "actions":["OPEN_DIAGNOSTICS","RUN_MIGRATION","RESTORE_BACKUP"],
  "issueReference":"DB-003",
  "supportBundleRecommended":true
}
```

Appendix B. Pseudocode and command reference

B.1 Supervisor startup pseudocode

Every await returns a structured result and preserves component logs

```
suspend fun start(profileId: String) = operation("runtime.start") {
    require(explicitUserAction)
    transition(PREFLIGHT)
    val report = preflight(profileId)
    if (!report.ok) fail(report.errors)

    beginSession(clean = false)
    foregroundNotification.show("Starting database")

    transition(DB_STARTING)
    database.start(session)
    database.awaitReadyAndCompatible(timeoutFor(DB))

    transition(REALM_STARTING)
    realm.start(session)
    realm.awaitReady(timeoutFor(REALM))

    transition(WORLD_STARTING)
    world.start(session, profileId)
    world.awaitReady(timeoutFor(WORLD, profileId))

    transition(CLIENT_STARTING)
    client.launch(session, profile.client)
    client.awaitWindow(timeoutFor(CLIENT))

    transition(RUNNING)
    persistJournal()
}
```

B.2 Supervisor stop pseudocode

Forced component termination must set clean=false

```
suspend fun stop(mode: StopMode) = operation("runtime.stop") {
    transition(STOPPING)
    client.requestClose().awaitBounded()

    if (world.isResponsive()) {
        world.command(SAVE).awaitBounded()
        world.command(SHUTDOWN_NOW).awaitBounded()
    }
    world.stopEscalatingIfNeeded()
    realm.stopEscalatingIfNeeded()
    database.checkpointAndStopEscalatingIfNeeded()

    val clean = world.cleanExit && database.cleanExit
    endSession(clean)
    transition(if (clean) STOPPED else RECOVERING)
    foregroundNotification.removeWhenAllStopped()
}
```

B.3 Recovery pseudocode

No repeated automatic recovery loop

```
suspend fun recoverIfNeeded(): RecoveryResult {
    val journal = loadJournal()
    val owned = discoverComponentsByPackageAndInstanceToken()
    stopOwnedOrphans(owned)

    if (journal.clean && owned.isEmpty()) return CLEAN

    val db = database.startRecoveryMode()
    return when {
        db.consistencyAndRevisionPass -> {
            database.stopCleanly()
            markRecovered(); RECOVERED
        }
        backups.hasCompatibleValidBackup -> NEEDS_RESTORE
```

```
        else -> BLOCKED_PRESERVE_EVIDENCE
    }
}
```

B.4 Development adb commands

Capture command output through the app where practical; these are development fallbacks

```
# Device/runtime facts
adb shell getprop ro.build.version.sdk
adb shell getprop ro.product.cpu.abilist
adb shell getprop ro.product.cpu.abilist32
adb shell getprop ro.product.cpu.abilist64
adb shell getconf PAGE_SIZE
adb shell dumpsys meminfo <package>
adb shell dumpsys thermalservice
adb shell df -h

# App/process/service state
adb shell dumpsys activity services <package>
adb shell ps -A | grep <package>
adb shell cat /proc/net/tcp
adb logcat --pid=<pid>

# Package/native artifacts
adb shell pm path <package>
adb shell dumpsys package <package>
zipalign -c -P 16 -v 4 app-release.apk
readelf -h -l -d libSomething.so

# Emulator host address reminder
# 127.0.0.1 = Android guest; 10.0.2.2 = Windows host loopback
```

B.5 CMaNGOS administrative command examples

Conceptual examples; verify exact pinned-core syntax

```
account create <USERNAME> <PASSWORD>
account set password <USERNAME> <NEWPASSWORD> <NEWPASSWORD>
account set gmlevel <USERNAME> <LEVEL>
server save
server shutdown 0

# Exact syntax and available commands must be taken from the pinned core.
# Never concatenate untrusted input into a shell command; use the structured control module.
```

B.6 Native dependency inspection

Use NDK LLVM tools from the pinned toolchain

```
file <artifact>
readelf -h <artifact>
readelf -d <artifact> | grep NEEDED
readelf -l <artifact> | grep -A2 LOAD
nm -D --defined-only <artifact>
llvm-readobj --needed-libs --program-headers <artifact>

# Fail CI if a target artifact references a host path/library or has wrong ELF machine.
# Preserve build ID and separate symbols for every shipped native object.
```

B.7 Import path normalization rules

Do not expose raw provider paths to native tools

```
normalize(relative):
  reject NUL and control characters
  replace provider separators with '/'
  reject absolute paths, drive prefixes, '..' segments and empty traversal
  Unicode-normalize according to documented policy
  compute caseFoldKey for collision detection
  reject special file/device/symlink escaping tree
  enforce maximum depth, component length, file count and file size
  return canonical relative path only
```

Appendix C. Emulator and device laboratory checklist

C.1 Windows host checklist

Item	Record/verify
CPU/virtualization	Host CPU model, cores, virtualization enabled, Hyper-V/WHPX/selected accelerator.
RAM/storage	Host available RAM and SSD free space; avoid host swapping during profile runs.
Android tools	Android Studio, emulator, SDK, adb versions; exact system image package IDs.
GPU	Host GPU/driver and emulator graphics mode.
Network	VPN/firewall/proxy state; host bridge tests labeled.
Time	Host and guest clocks; use monotonic timestamps for measurements.
Client source	User-owned build-5875 directory outside repository/artifacts.

C.2 AVD creation record

Check this record into test/avd

```
avdId: AVD-Modern-x86_64-v1
systemImage: system-images;android-<api>;<tag>;x86_64
emulatorVersion: <version>
ramMiB: 8192
vmHeapMiB: <value>
dataPartitionGiB: 32
graphics: hardware/angle/host <exact>
cores: <count>
pageSize: 4096 or 16384
abiList: <captured>
snapshotPolicy: cold boot for release tests
networkCondition: offline/local-only unless test states otherwise
```

C.3 First emulator run checklist

ID	Check
C-01	Cold boot and capture capability report before installing app.
C-02	Install exact APK and record SHA-256/signing certificate.
C-03	Grant only requested notifications/SAF access; no root.
C-04	Run package-native self-tests and export bundle.
C-05	Import from a normal guest Documents folder through SAF.
C-06	Disable host network after setup and prove local play.
C-07	Take no AVD snapshot during active DB/world write.
C-08	For reproducible release tests, wipe data or restore a known stopped-state snapshot.
C-09	Capture host CPU/GPU utilization to detect emulator/host bottleneck.
C-10	Repeat on 16 KB AVD before release candidate.

C.4 Physical ARM device record

Field	Example content
Device/build	Manufacturer/model, Android build fingerprint, security patch.
CPU/ABI	SoC, cores, abilist32/64, 64-bit-only status.
Memory	Physical RAM, swap/zram, memory class, observed available at start.
GPU	Vendor/renderer, Vulkan/OpenGL versions, driver/Mesa profile.

Field	Example content
Page size	4096 or 16384.
Thermal/power	Battery capacity, charging mode, ambient range.
Runtime	Box/Wine/DXVK/Turnip IDs and selected branch.
Qualified profiles	Resolution/FPS/bots and pass dates.

C.5 Measurement rules

- Warm-up shader/runtime caches separately from cold-start measurements; publish both.
- Use the same character, route, time of day and bot profile for comparative performance scenes.
- Record whether the device is charging, case on/off, ambient temperature and starting battery percentage.
- Do not compare emulator FPS to ARM FPS as equivalent hardware. Compare regressions within each reference profile.
- A result without app/runtime manifest ID is not admissible in the performance database.

Appendix D. Community evidence and lessons

How to read this appendix

Community reports are valuable for discovering failure modes and workable combinations, but they are not controlled benchmarks. Every item below is converted into an engineering hypothesis or test. The report does not treat anecdotal frame rates or device claims as guarantees.

ID	Observation	Engineering interpretation
D-01	singlePlayerWow-android demonstrates an Android-native server in Termux and a separate Winlator client.	The split server/client concept is practical, but the project is WotLK/AzerothCore-oriented, manual, and not a one-APK Vanilla implementation. [S11]
D-02	Its scripts install Clang/CMake/MariaDB dependencies and contain Android-specific build workarounds.	Expect dependency discovery, linker naming, memory and version-check issues; replace workarounds with pinned build fixes. [S12]
D-03	The repository's current issue list includes setup drift such as addon versions, realm-list loops and directory problems.	Pin all dependencies, detect common path/address errors, and provide stage-specific repair. [S35]
D-04	A Reddit demonstration describes offline WoW with bots using a Termux server plus Winlator client.	Use as proof of demand/possibility; reproduce with the selected Vanilla stack and integrated supervisor. [S20]
D-05	Some Winlator users report reaching login but disconnecting.	Create separate auth/realm/world readiness tests; do not diagnose every disconnect as Wine failure. [S21]
D-06	Reports suggest clients/servers without modern launchers or anti-cheat are more practical under Winlator.	Require direct WoW.exe launch and reject unknown launcher/injector dependency in the supported path. [S22]
D-07	Users report large touch overlays and difficult screen real estate.	Measure visible play area and pass touch-only tasks; allow compact/custom layouts. [S23]
D-08	A phone setup report highlights chat and precise target selection as difficult.	Build an explicit IME bridge and target/cursor mode; include UX-T03/T06. [S24]
D-09	CMaNGOS issue reports describe Classic playerbot group-AI problems.	Pin the bot commit and turn target/pull/loot behavior into scripted regressions. [S25]
D-10	Map/vmap/mmap documentation and issues show multiple generated datasets and CPU-intensive generation choices.	Version and validate each stage; checkpoint mmaps and publish a coverage report. [S31, S32]
D-11	ShaguController provides a Vanilla 1.12 controller-friendly addon but still expects external key mapping.	Use it only as optional UI support; Android remains responsible for input synthesis. [S28]
D-12	Termux tracked the API-29 writable-home executable restriction years ago.	Do not mistake a target-28 hobby stack for a compliant modern application architecture. [S13, S33]

D.1 Community claims that must be re-measured

- Frame rates, battery drain and thermal behavior on any named phone.
- Maximum usable random-bot population.
- Which Winlator/Wine/DXVK/driver release is "best."
- Whether a specific login disconnect is caused by client, realm address, build or server configuration.
- Whether a given addon package is truly Vanilla 1.12-compatible.
- How long extraction or first bot generation takes.

Appendix E. Source register

Accessed 1 August 2026. Repository behavior is tied to the cited page/commit where shown; the implementation must pin its own exact release commits. AUTH = official documentation/upstream source; CODE = repository implementation evidence; COMM = community report.

S01 [CODE] SPP Classics CMaNGOS repository and README — https://github.com/celguar/spp-classics-cmangos

S02 [CODE] SPP Classics release workflow — https://github.com/celguar/spp-classics-cmangos/releases

S03 [AUTH] CMaNGOS Classic upstream repository and GPL license — https://github.com/cmangos/mangos-classic

S04 [AUTH] CMaNGOS installation instructions — https://github.com/cmangos/issues/wiki/Installation-Instructions

S05 [CODE] CMaNGOS RealmList.cpp supported client builds, including 5875 — https://github.com/cmangos/mangos-classic/blob/master/src/realmd/RealmList.cpp

S06 [CODE] CMaNGOS mangosd configuration: database sockets, port 8085 and bind defaults — https://github.com/cmangos/mangos-classic/blob/master/src/mangosd/mangosd.conf.dist.in

S07 [AUTH] CMaNGOS playerbots repository and feature overview — https://github.com/cmangos/playerbots

S08 [CODE] CMaNGOS playerbot command reference — https://github.com/cmangos/mangos-classic/blob/master/doc/PlayerBot/commands.txt

S09 [AUTH] Winlator upstream repository, Wine/Box architecture and LGPL-2.1 — https://github.com/brunodev85/winlator

S10 [AUTH] Box64 README: Wine64, Box86 requirement for ordinary 32-bit components and experimental WoW64 route — https://github.com/ptitSeb/box64/blob/main/README.md

S11 [CODE] singlePlayerWow-android: native Termux server plus Winlator client proof — https://github.com/duall/singlePlayerWow-android

S12 [CODE] singlePlayerWow-android cutoff build script and Android workarounds — https://github.com/duall/singlePlayerWow-android/blob/main/wowsp_cutoff.sh

S13 [AUTH] Android 10/API 29 behavior changes: execute permission removed from writable app home — https://developer.android.com/about/versions/10/behavior-changes-10

S14 [AUTH] Android 16 KB page-size support and test/build guidance — https://developer.android.com/guide/practices/page-sizes

S15 [AUTH] Android Storage Access Framework and directory access — https://developer.android.com/training/data-storage/shared/documents-files

S16 [AUTH] Android 14 foreground-service types, including specialUse — https://developer.android.com/about/versions/14/changes/fgs-types-required

S17 [AUTH] Android foreground-service timeouts — https://developer.android.com/develop/background-work/services/fgs/timeout

S18 [AUTH] Android process and application lifecycle — https://developer.android.com/guide/components/activities/process-lifecycle

S19 [AUTH] Android Emulator network address space, including 10.0.2.2 — https://developer.android.com/studio/run/emulator-networking-address

S20 [COMM] Reddit: offline WoW with bots using Android server plus Winlator client — https://www.reddit.com/r/winlator/comments/1mco5vx/you_can_play_wow_offline_with_bots_singleplayer/

S21 [COMM] Reddit: WoW on Winlator login/disconnect discussion — https://www.reddit.com/r/EmulationOnAndroid/comments/15k6zki/world_of_wacraft_on_winlator/

S22 [COMM] Reddit: WoW client/server without launcher or anti-cheat on Winlator — https://www.reddit.com/r/EmulationOnAndroid/comments/1obg0b6/run_wow_classic_on_winlator_server_without/

S23 [COMM] Reddit: WoW mobile overlay/screen-space discussion — https://www.reddit.com/r/winlator/comments/1k83dpf/just_to_prove_its_100_online_wow_cata_and_wow/

S24 [COMM] Reddit: Galaxy phone WoW/Winlator setup, controls/chat observations — https://www.reddit.com/r/EmulationOnAndroid/comments/1subs3r/my_world_of_warcraft_setup_samsung_galaxy_s25_on/

S25 [COMM] CMaNGOS issue #1648: Classic PlayerBot group AI issues — https://github.com/cmangos/issues/issues/1648

S26 [COMM] SinglePlayerProject community discussions used as pathing/LOS hypotheses — https://www.reddit.com/r/SinglePlayerProject/

S27 [CODE] SPP Classics bot configuration/default warnings (same repository as S01) — https://github.com/celguar/spp-classics-cmangos

S28 [AUTH] ShaguController: controller-friendly UI for Vanilla WoW 1.12 — https://github.com/shagu/ShaguController

S29 [AUTH] Android NDK ABI documentation — https://developer.android.com/ndk/guides/abis

S30 [AUTH] Android Emulator hardware/graphics acceleration — https://developer.android.com/studio/run/emulator-acceleration

S31 [CODE] MaNGOS wiki: extracting DBC/maps/vmaps/mmaps — https://github.com/mangoswiki/Wiki/blob/master/library/Installation%20Guides/General/Extracting-Game-Assets.md

S32 [COMM] CMaNGOS issue #1877: high-resolution extraction and MoveMap generation — https://github.com/cmangos/issues/issues/1877

S33 [COMM] Termux issue #1072 documenting target-API writable-home exec restriction — https://github.com/termux/termux-app/issues/1072

S34 [AUTH] Android ApplicationInfo native-library extraction flag/reference — https://developer.android.com/reference/android/content/pm/ApplicationInfo

S35 [COMM] singlePlayerWow-android issue tracker: dependency/setup drift examples — https://github.com/duall/singlePlayerWow-android/issues

S36 [AUTH] Android foreground-service launch restrictions — https://developer.android.com/develop/background-work/services/fgs/launch

S37 [CODE] Winlator build configuration at examined commit — https://github.com/brunodev85/winlator-app/blob/c2f4ad4534f4637b543a9a3b085e28f50cf6d01c/app/build.gradle

S38 [AUTH] Wine licensing — https://www.winehq.org/license

S39 [AUTH] Box64 upstream repository and MIT license — https://github.com/ptitSeb/box64

S40 [AUTH] MariaDB licensing FAQ — https://mariadb.com/kb/en/licensing-faq/

Appendix F. Glossary, decisions and hand-off checklist

F.1 Glossary

Term	Definition
ABI	Application Binary Interface; Android native code target such as x86_64 or arm64-v8a.
AVD	Android Virtual Device configuration used by the Android Emulator.
Box86/Box64	Dynamic CPU translation runtimes used to run x86/x86-64 Linux code on non-x86 hosts, commonly paired with Wine.
CMaNGOS	Continued MaNGOS open-source World of Warcraft server emulator project.
DBC	Client data tables extracted for server use.
DXVK	Direct3D-to-Vulkan translation layer commonly used with Wine.
FGS	Android foreground service: user-visible long-running work with notification and policy constraints.
Managed client	App-owned copy of user-supplied WoW files used at runtime.
MMap	Movement map/navigation data used for pathfinding.
PSS/RSS	Memory accounting measures; PSS apportions shared pages, RSS counts resident pages.
SAF	Android Storage Access Framework used for user-granted document/folder access.
SPP Classics	Windows-oriented Single Player Project repack/workflow used here as a functional reference.
Turnip	Open-source Vulkan driver commonly used for Qualcomm Adreno in Android Windows-compatibility stacks.
VMap	Collision/line-of-sight geometry data used by the server.
Wine	Compatibility layer that runs Windows applications by implementing Windows APIs.
WineD3D	Wine Direct3D translation commonly targeting OpenGL.
WoW64	Wine/Windows architecture enabling 32-bit Windows applications under a 64-bit environment.
World-ready	Structured signal that the server completed required loading and can accept sessions.

F.2 Decisions the receiving agent must not reopen casually

- Do not attempt to recompile or rewrite the proprietary client as the first path.
- Do not bundle client or derived proprietary data.
- Do not run the Windows SPP server stack under Wine.
- Do not base production on target-28 writable-directory execution.
- Do not treat stock Winlator as an x86 emulator solution.
- Do not enable bots before zero-bot persistence and recovery pass.
- Do not use 10.0.2.2 in the integrated release configuration.
- Do not expose database/admin sockets beyond the app sandbox.
- Do not update unpinned core/database/playerbot components independently.
- Do not call a session successful until touch usability, clean stop and forced recovery are tested.

F.3 Hand-off checklist

ID	Hand-off evidence
H-01	Repository contains ADR-001–012 and compatibility lock template.
H-02	Reference AVD definitions and capability reports are checked in.
H-03	User-owned client files are outside source control/build artifacts.
H-04	Production packaging spike has a written result on current target and 16 KB AVD.
H-05	x86 direct-Wine and native-DB proofs have independent reproducible scripts/tests.

ID	Hand-off evidence
H-06	All upstream commits, patches and licenses are recorded.
H-07	Issue IDs and structured error codes are wired into logs/test reports.
H-08	Performance results identify exact device/runtime/profile.
H-09	Every destructive operation has backup/rollback and user confirmation.
H-10	Final release claim lists exact supported client build, ABIs, GPUs, RAM and bot tiers.

F.4 Final implementation directive

> **Build the system as a set of replaceable, measured contracts**
>
> The successful solution is not “make Winlator launch SPP.” It is an Android product whose importer, native local realm, account/bot control, Windows-client runtime, input layer and recovery system each have a precise contract and independent proof. Establish the x86 client and native database proofs first, integrate with bots off, and allow the ARM backend to change without touching world data or user workflow.